

Vietnam National University
Ho Chi Minh city University of Technology
Faculty of Computer Science and Engineering



ELECTRONIC COMMERCE
Database Systems(CO2014) - CC09

Report Date: 07/10/2025 - Semester 251

Lecturer: NGUYEN MINH TAM

Student name	Student ID	Email
Đỗ Đăng Khoa	2352554	khoa.do250805@hcmut.edu.vn
Trần Liên Khang	2352504	khang.tran0209@hcmut.edu.vn
Tào Nguyễn Quang Khang	2352499	khang.tao206bkuhcm@hcmut.edu.vn
Nguyễn Phước Quý Bảo	2352100	bao.nguyen090105@hcmut.edu.vn

Table of Contents

Table of Contents	1
List of Figures	5
List of Tables	6
1 Introduction	7
2 Application Description	7
2.1 Research related applications/systems	7
2.2 Description of the Proposed System	8
2.2.1 General Description	8
2.2.2 Detailed Description of Entity Types, Attributes and Relationships	9
3 Description of Semantic Constraints	12
3.1 ACCOUNT	12
3.2 CUSTOMER	12
3.3 SHOP	12
3.4 ADMIN	12
3.5 CATEGORY	12
3.6 PRODUCT	12
3.7 PRODUCTITEM	13
3.8 CART	13
3.9 ORDER	13
3.10 ORDERITEM	13
3.11 VOUCHER	13
3.12 SHIPPING	13
3.13 REVIEW	13
4 Entity Relationship Diagram (ERD)	14
5 Relational Database Schema	14
5.1 Mapping Diagram	14
5.2 Relational Database Schema Description	15
6 Selected Technology	17
6.1 Technology Stack	17

6.1.1	Database Management System	17
6.1.2	Backend Framework	17
6.1.3	Frontend Framework	17
6.1.4	Authentication	18
7	Database Implementation	18
7.1	Account Management	18
7.1.1	ACCOUNT Table	18
7.1.2	CUSTOMER Table	18
7.1.3	SHOP Table	19
7.1.4	ADMIN Table	19
7.2	Product Catalog	20
7.2.1	CATEGORY Table	20
7.2.2	PRODUCT Table	20
7.2.3	PRODUCTITEM Table	21
7.3	Shopping Cart	21
7.3.1	CART Table	21
7.3.2	CARTITEM Table	22
7.4	Orders and Items	22
7.4.1	ORDER Table	22
7.4.2	ORDERITEM Table	23
7.5	Promotions	23
7.5.1	VOUCHER Table	23
7.6	Shipping	24
7.6.1	SHIPPING Table	24
7.7	Reviews and Feedback	24
7.7.1	REVIEW Table	24
8	Sample Data Insertion	25
8.1	Account Data	25
8.2	Customer Data	25
8.3	Shop Data	26
8.4	Category Data	26
8.5	Product and ProductItem Data	27
8.6	Shipping and Voucher Data	28
8.7	Order Transaction Data	29
8.8	Review Data	30
9	Functions and Procedures	31
9.1	Account Management Functions	31
9.1.1	Function: Calculate Customer Total Spent	31
9.2	Shop Management Functions	32

9.2.1	Function: Calculate Shop Revenue	32
9.2.2	Function: Calculate Shop Rating	34
9.3	Product Management Functions	35
9.3.1	Function: Calculate Product Rating	35
9.3.2	Function: Calculate Order Total	36
9.3.3	Function: Get Order Total Items with Cursor	38
9.4	Category Hierarchy Procedures	40
9.4.1	Procedure: Get Category Hierarchy (All Levels)	40
9.4.2	Procedure: Get All Children of a Category	41
9.4.3	Procedure: Get All Parents of a Category	43
9.5	Order Management Procedures	44
9.5.1	Procedure: Create Order from Cart	44
9.5.2	Procedure: Apply Voucher	49
9.6	Shop Management Procedures	51
9.6.1	Procedure: Get Shop Revenue Report	51
9.7	Product CRUD Procedures	53
9.7.1	Procedure: Add Product	53
9.7.2	Procedure: Update Product	55
9.7.3	Procedure: Delete Product	57
9.7.4	Procedure: Add Product Item (Variant)	58
9.7.5	Procedure: Update Product Item	60
9.7.6	Procedure: Delete Product Item	61
9.8	Product Search Procedure	62
9.8.1	Procedure: Get Product List with Filters	62
10	Database Triggers	65
10.1	Order Management Triggers	66
10.1.1	Trigger: Order Item Before Insert	66
10.1.2	Trigger: Order Item After Insert	67
10.1.3	Trigger: Order After Update (Stock Restoration)	69
10.2	Review Management Triggers	71
10.2.1	Trigger: Review Before Insert (Verified Purchase)	71
10.2.2	Trigger: Review After Insert (Rating Update)	72
10.3	Inventory Management Triggers	74
10.3.1	Trigger: Product Item Before Update	74
10.4	Account Management Triggers	75
10.4.1	Trigger: Account After Insert (Subclass Creation)	75
10.4.2	Trigger: Customer Before Insert (Code Generation)	76
10.4.3	Trigger: Shop Before Insert (Code Generation)	77
11	Application Demo and System Workflow	78

11.1	System Architecture Overview	78
11.1.1	Three-Tier Architecture Design	78
11.2	Main Features Demonstration	80
11.2.1	Feature 1: Product Search and Discovery	80
11.2.2	Feature 2: Shopping Cart Management	80
11.2.3	Feature 3: Order Creation with Voucher	81
11.2.4	Feature 4: Product Review and Rating	82
11.2.5	Feature 5: Shop Revenue Analytics	83
12	References and Video Demonstration	83
12.1	GitHub Repository	83
12.2	Video Demonstration	83
13	Conclusion	83
13.1	Summary	84
13.2	Key Features	84

List of Figures

1	Homepage interface of Shopee when accessing the website for the first time . . .	8
2	Product detail page with variant selection (size, color), price display, and image gallery	8
3	ERD diagram	14
4	Relational Schema Diagram	15

List of Tables

1	Mapping to Database Schema	15
---	--------------------------------------	----

1 Introduction

In the era of digital commerce, online marketplaces have become an essential medium for modern retail activities. Platforms such as Shopee demonstrate the effectiveness of connecting millions of buyers and sellers through a unified digital infrastructure, providing convenience, scalability, and automation in commercial transactions. Inspired by this model, the proposed system aims to design a database to support a multi-role e-commerce platform in which individual customers can browse and purchase products, merchants can manage storefronts and process orders, and system administrators can supervise platform operations.

This report focuses on analyzing the functional requirements of such a platform and designing a suitable database structure to support core activities, including user account management, product categorization, shopping cart handling, order processing, voucher application, shipping coordination, and post-purchase feedback. The objective is to construct a logical data model that ensures consistency, scalability, and extensibility, laying the foundation for future development of a complete e-commerce application.

2 Application Description

2.1 Research related applications/systems

The selected reference application is **Shopee**, one of the leading E-Commerce platforms in Vietnam. The website can be accessed at <https://shopee.vn>. **Shopee** connects customers and sellers in an online marketplace, providing a comprehensive system to browse, purchase, and manage products. Its business process begins with account registration, where users can sign up as customers or sellers. Customers browse products organized in hierarchical categories, select variants of products such as size or color, add items to a shopping cart, and proceed to checkout with payment and shipping options. Sellers manage shop profiles, upload and update product listings, track inventory, and monitor customer reviews to improve service quality.

A key feature of **Shopee** is the multi-shop order system, where a single order can include products from multiple sellers, each tracked individually in the backend. Another important element is the voucher system, which applies discounts based on conditions such as minimum order value or expiration dates. **Shopee** also integrates a review and rating system, ensuring that verified purchases can leave feedback on the product and the store, which helps improve trust and transparency.

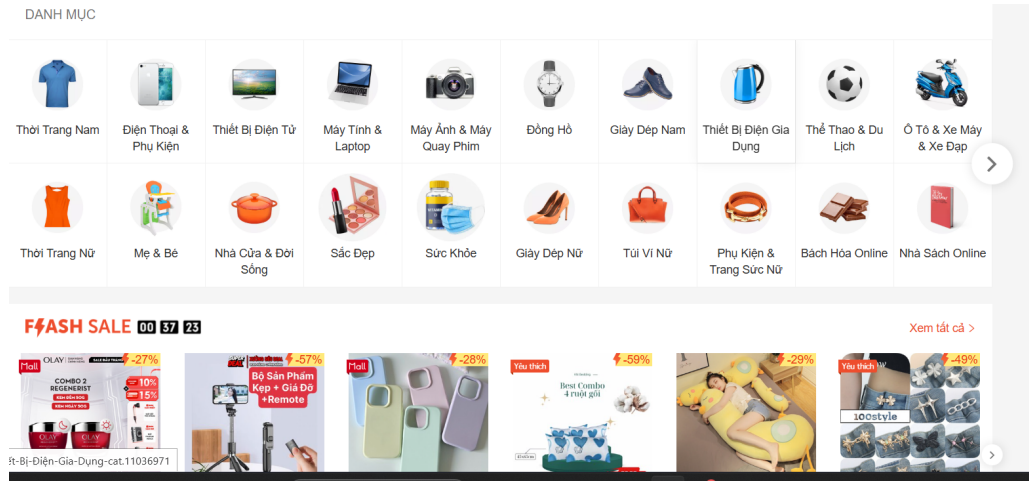


Figure 1: Homepage interface of **Shopee** when accessing the website for the first time

The homepage highlights featured campaigns, recommended products, and navigation menus, such as categories, products and flash sales. This layout indicates that **Shopee** focuses heavily on visual-driven marketing and personalized product discovery.

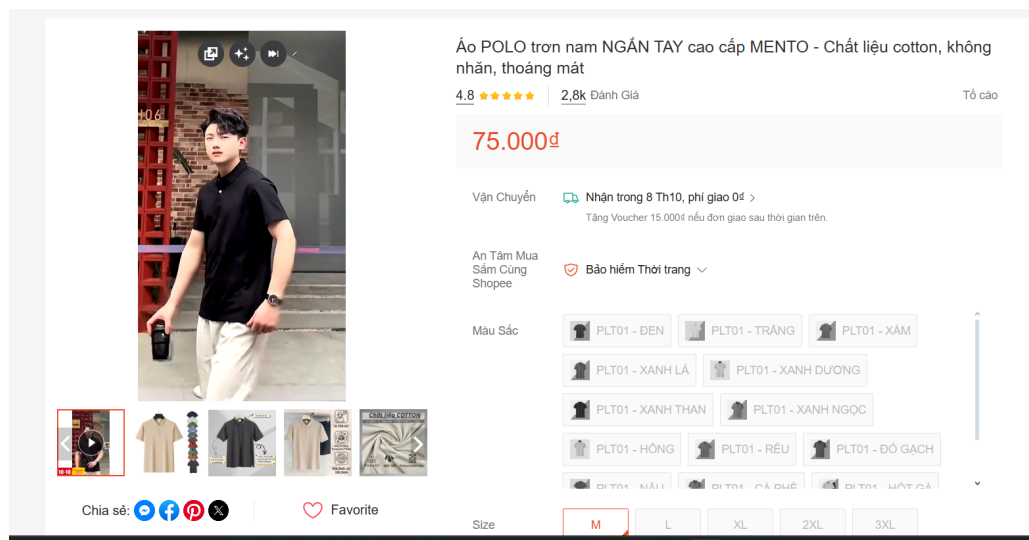


Figure 2: Product detail page with variant selection (size, color), price display, and image gallery

This screen demonstrates the purchasing flow, where users can select a specific **product variant**, adjust quantity, add to cart or directly place an order. Essential information such as ratings, shipping options, and shop reputation is also displayed to support decision-making.

2.2 Description of the Proposed System

2.2.1 General Description

The proposed system is an e-commerce platform inspired by Shopee, designed to support online retail activities between buyers and sellers. It enables individuals and businesses to create selling

accounts, publish products, and manage orders, while customers can browse, purchase, and evaluate products in a seamless digital marketplace.

Types of Users: **Customer:** A regular user who browses products, adds items to their shopping cart, places orders, applies vouchers, selects shipping methods, and provides product or shop reviews after successful purchases.

Shop: A seller entity responsible for managing product listings, defining product variants, monitoring stock levels, handling orders, and receiving feedback from customers.

Admin: A privileged system user who supervises overall platform operations, manages user accounts, resolves disputes, and moderates inappropriate content when necessary.

Main System Functions: For **Customers:** Browse products by category, search and filter based on attributes, add items to cart, update quantities, place orders with preferred shipping and payment methods, apply vouchers, and submit verified reviews for purchased products.

For **Shops:** Register as sellers, manage store profile, create product listings with multiple variants, track inventory, process incoming orders, and analyze ratings and feedback to improve service quality.

For **Admins:** Oversee account registration activities, suspend violating accounts, manage system-wide configurations such as shipping options and voucher policies, and intervene in reported cases of fraud or misconduct.

2.2.2 Detailed Description of Entity Types, Attributes and Relationships

ACCOUNT Entity The system maintains a centralized **ACCOUNT** entity to identify every registered user. Each account is uniquely defined by an **AccountID** (auto-incremented primary key) and stores essential authentication details such as **Email** and **Password**. Personal identification is represented through the attribute **FullName**, while **Phone** stores the primary contact information and must follow Vietnamese phone format (10 digits). The **Role** field classifies each account as Customer, Shop, or Admin, ensuring proper access control. The attribute **Status** maintains activation state with values 'Active' or 'Ban' for moderation purposes. **CreatedAt** records the registration timestamp automatically.

CUSTOMER Entity The **CUSTOMER** entity is a specialization of **ACCOUNT** and represents end-users participating in purchasing activities. Each customer is referenced by a distinct **CustomerID** (same as AccountID) and an auto-generated **CustomerCode** with 'CUST' prefix (CUST00001, CUST00002, ...). Customers have an **Address** for default shipping destination and **AddPhone** for additional contact. The system maintains **TotalSpent** and **TotalOrder** as derived attributes to support customer ranking and loyalty programs. Each **CUSTOMER** owns exactly one **CART**.

SHOP Entity The **SHOP** entity models registered merchants. Each shop is assigned a unique **ShopID** (same as AccountID) and an auto-generated **ShopCode** with 'SHOP' prefix (SHOP00001, SHOP00002, ...). A shop operates under a **ShopName** and has a **ShopPhone** for business inquiries. The **AddressShop** identifies its operational base, while **Rating** reflects aggregated customer

feedback (0-5 scale). The **ShopStatus** attribute can be 'Open', 'Temporarily Close', or 'Closed'. A single SHOP may publish multiple **PRODUCTS**.

ADMIN Entity The **ADMIN** entity represents internal supervisory staff. Each admin is identified by an **AdminID** (same as AccountID) and is linked to an **AccountID**. The **Role** field specifies the admin's level of access (Core, Moderator, or Support), ensuring proper authorization. Unlike other roles, admins do not engage in transactions but retain full access rights for system governance. **Note** field allows additional admin information.

CATEGORY Entity The **CATEGORY** entity organizes products into a navigable structure. Each category has a unique **CategoryID** and a descriptive **CategoryName** (max 100 characters). Categories support hierarchical organization through **ParentCategoryID**, which can be null for root categories. This recursive 1:N relationship enables multi-level product browsing.

PRODUCT Entity The **PRODUCT** entity represents commercial goods listed by shops. Each product is uniquely defined by a **ProductID**, belongs to exactly one SHOP through **ShopID**, and must be classified under a CATEGORY via **CategoryID**. Descriptive attributes include **ProductName**, **Description**, **Image**, and **CreatedAt**. The attribute **Status** specifies whether the product is 'In stock' or 'Out of stock'. **Rating** is auto-calculated from customer reviews. Products serve as parent entities for **PRODUCTITEM** (variants).

PRODUCTITEM Entity **PRODUCTITEM** is a weak entity that models purchasable variations of a product. Each item is uniquely identified by **ItemID** and belongs to a specific **ProductID**. Each variant contains attributes such as **Color**, **Type** (size, configuration), **Price**, **Stock**, and optionally **ImageURL** for representation. Variants enable customers to differentiate between sizes, configurations, or styles within the same product family. **ShopID** is stored for quick reference to the selling shop.

CART Entity The **CART** entity holds temporary selections before checkout. Each cart is exclusively owned by one CUSTOMER and distinguished by **CartID**. The relationship between CART and PRODUCTITEM is modeled as a many-to-many association through **CARTITEM**, enriched with the attribute **Quantity**. **CreatedAt** and **UpdatedAt** track cart lifecycle.

CARTITEM Entity **CARTITEM** is a junction table representing the many-to-many relationship between CART and PRODUCTITEM. Each entry is identified by **CartItemID** and contains **Quantity** (the number of units chosen for each specific variant).

ORDER Entity The **ORDER** entity records finalized purchases. Each order is uniquely identified by an **OrderID** and linked to a CUSTOMER via **CustomerID**. Transaction attributes include **OrderDate**, **ShippingAddress**, and **PaymentMethod** (COD, Banking, Momo, ZaloPay). The system computes **TotalAmount** automatically based on selected variants and applied promotions. Each order references exactly one **SHIPPING** through **ShippingID** and may optionally apply a **VOUCHER** through **VoucherID**. The **Status** attribute tracks fulfillment progress ('Processing', 'Shipped', 'Delivered', 'Cancelled'). **DeliveredDate** records when the order was completed.

ORDERITEM Entity **ORDERITEM** is a weak entity representing individual product lines within an order. It is uniquely identified by composite key (**OrderID**, **ItemID**) and further references **ShopID** to determine the responsible seller. Each item records **Quantity** and **PriceAtPurchase** to preserve historical pricing data regardless of later modifications.

VOUCHER Entity The **VOUCHER** entity regulates promotional campaigns. Each voucher is characterized by **VoucherID**, **Code** (unique voucher code), **DiscountType** (Percentage or Amount), **DiscountValue** (discount amount/percentage), **MinOrderValue** (minimum order value to apply), **StartDate**, and **ExpiredDate**. **UsageLimit** controls maximum uses, and **UsedCount** tracks actual uses. **Status** indicates if the voucher is 'Active' or 'Expired'. Vouchers can be applied to orders subject to constraint checks.

SHIPPING Entity The **SHIPPING** entity defines available delivery options. Each shipping method is identified by **ShippingID** and described by **Name**, **EstimatedDays**, and **Fee**. Orders rely on this entity to determine logistics partners. **Status** can be 'Active' or 'Inactive'.

REVIEW Entity The feedback system incorporates a unified **REVIEW** entity to handle both product and shop reviews. A **REVIEW** is identified by **ReviewID** and associated with **CustomerID** and **TargetType** (either 'Product' or 'Shop'). **TargetID** indicates which product or shop is being reviewed. It stores evaluation details such as **Rating** (1-5), **Comment**, optional **ImageURL**, and **ReviewDate**. Only customers who have previously purchased from the seller (verified purchase) may submit such reviews.

3 Description of Semantic Constraints

3.1 ACCOUNT

- **Email** and **Password** fields are mandatory for all accounts, as they are required for authentication.
- The **Role** field can only accept one of the following values: Customer, Shop, or Admin, ensuring clear role-based access control.
- The **Status** field can only accept ‘Active’ or ‘Ban’. A banned account cannot perform transactions or access certain features.
- **Phone** must be exactly 10 digits following Vietnamese phone format.

3.2 CUSTOMER

- **CustomerCode** is auto-generated with ‘CUST’ prefix (CUST00001, CUST00002, ...).
- A customer can only leave **Review** for items or shops they have actually purchased from, enforcing a verified purchase policy.

3.3 SHOP

- **ShopCode** is auto-generated with ‘SHOP’ prefix (SHOP00001, SHOP00002, ...).
- **Rating** field must always be within the range 0 to 5, reflecting customer feedback.
- **ShopStatus** can only accept ‘Open’, ‘Temporarily Close’, or ‘Closed’.

3.4 ADMIN

- The **Role** field must specify the admin’s level of access: Core, Moderator, or Support.

3.5 CATEGORY

- **CategoryName** field is mandatory and must not exceed 100 characters.
- **ParentCategoryID** can be null for root categories; otherwise enforces hierarchical organization.

3.6 PRODUCT

- **ProductName** and **ShopID** fields are mandatory for proper identification and ownership.
- **Status** can only be ‘In stock’ or ‘Out of stock’, automatically updated by triggers when stock reaches zero.

3.7 PRODUCTITEM

- **Price** must be ≥ 0 , and **Stock** must be ≥ 0 to prevent invalid transactions.

3.8 CART

- **Quantity** must be ≥ 1 for each cart item.

3.9 ORDER

- **TotalAmount** cannot be negative, ensuring correct financial records.
- **Status** can only accept: Processing, Shipped, Delivered, or Cancelled.
- **DeliveredDate** must be $\geq$ **OrderDate** when set, ensuring chronological consistency.

3.10 ORDERITEM

- **Quantity** must be ≥ 1 to reflect the actual purchase.
- **PriceAtPurchase** must be ≥ 0 , capturing the correct price at transaction time..

3.11 VOUCHER

- **DiscountValue** field must be ≥ 0 .
- **DiscountType** field allows only 'Percentage' or 'Amount'.
- **MinOrderValue** and **UsageLimit** must be ≥ 0 and ≥ 1 respectively.
- **ExpiredDate** must be $\geq$ **StartDate**.

3.12 SHIPPING

- **EstimatedDays** must be ≥ 0 , ensuring logical delivery time estimation.
- **Fee** must be ≥ 0 .

3.13 REVIEW

- **Rating** field is mandatory and must be between 1 and 5.
- **TargetID** combined with **TargetType** uniquely identifies the reviewed entity.

4 Entity Relationship Diagram (ERD)

This is the Entity Relationship Diagram: [HERE](#)

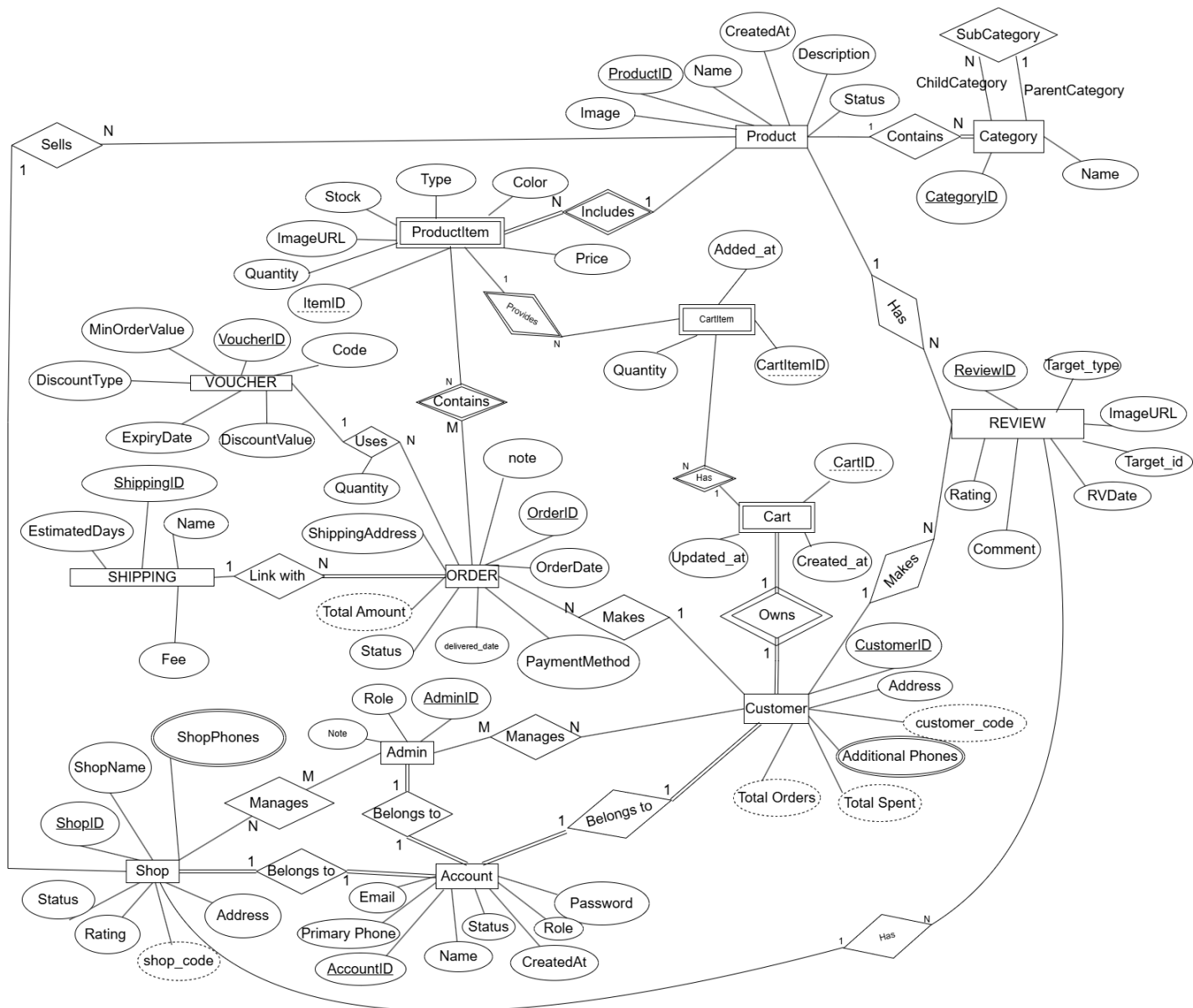


Figure 3: ERD diagram

5 Relational Database Schema

5.1 Mapping Diagram

This is the Relational Database Schema Diagram: [HERE](#)

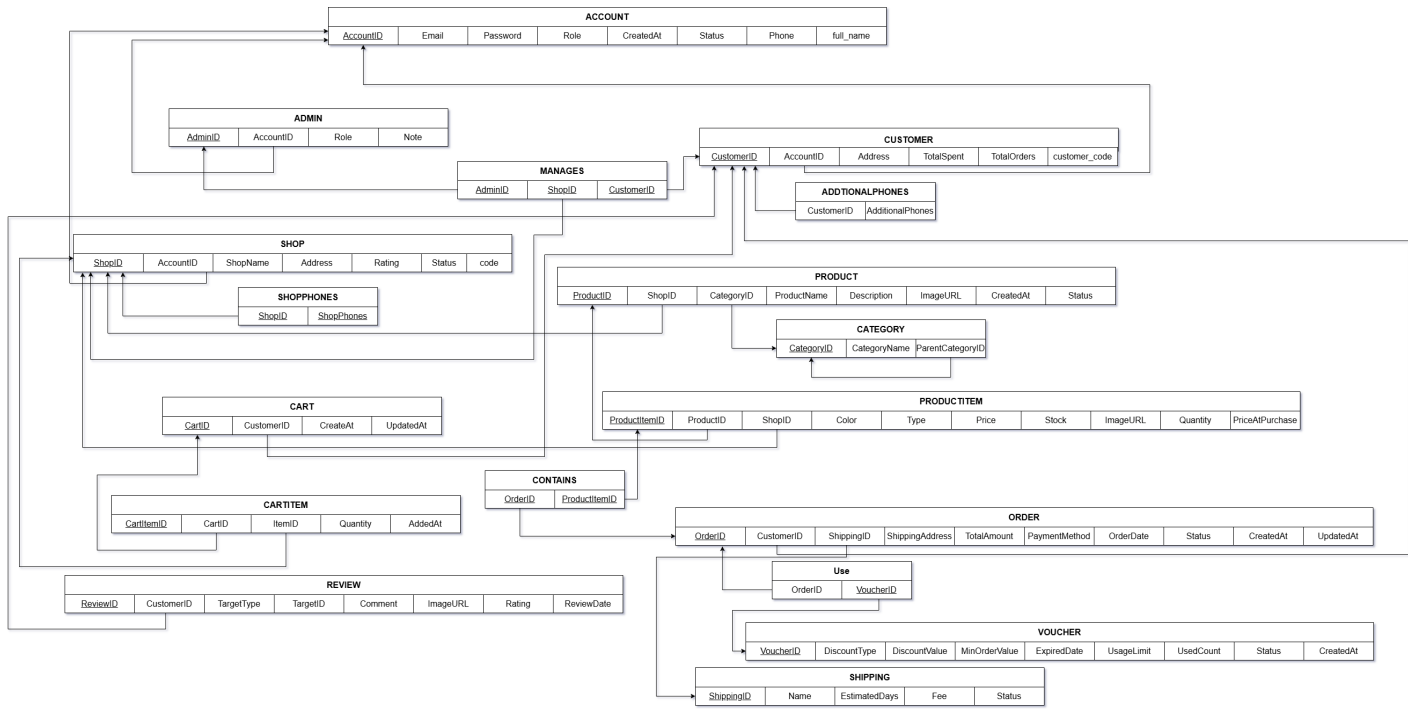


Figure 4: Relational Schema Diagram

5.2 Relational Database Schema Description

Table 1: Mapping to Database Schema

Name	Primary Key	Foreign Key
ACCOUNT	account_id	—
CUSTOMER	customer_id	customer_id → ACCOUNT.account_id
SHOP	shop_id	shop_id → ACCOUNT.account_id
ADMIN	admin_id	admin_id → ACCOUNT.account_id
CATEGORY	category_id	parent_category_id → CATEGORY.category_id
PRODUCT	product_id	shop_id → SHOP.shop_id; category_id → CATEGORY.category_id
PRODUCT-ITEM	item_id	product_id → PRODUCT.product_id; shop_id → SHOP.shop_id
CART	cart_id	customer_id → CUSTOMER.customer_id
CARTITEM	cart_item_id	cart_id → CART.cart_id; item_id → PRODUCTITEM.item_id
SHIPPING	shipping_id	—
VOUCHER	voucher_id	—
ORDER	order_id	customer_id → CUSTOMER.customer_id; shipping_id → SHIPPING.shipping_id; voucher_id → VOUCHER.voucher_id

Continued on next page

Table 1 – *Continued from previous page*

Name	Primary Key	Foreign Key
ORDERITEM	order_item_id	order_id → ORDER.order_id; item_id → PRODUCTITEM.item_id; shop_id → SHOP.shop_id
REVIEW	review_id	customer_id → CUSTOMER.customer_id

6 Selected Technology

6.1 Technology Stack

The proposed e-commerce platform utilizes a modern, scalable technology stack designed to support concurrent users, complex transactions, and real-time data synchronization:

6.1.1 Database Management System

MySQL Server 8.0 is selected as the primary DBMS due to its:

- **ACID Compliance:** Ensures data integrity across multi-seller transactions
- **Foreign Key Constraints:** Enforces referential integrity across 14 entities
- **Triggers and Stored Procedures:** Enables business logic automation (auto-generated codes, inventory management, rating calculations)
- **Recursive Queries:** Supports hierarchical category traversal
- **JSON Support:** Optional future extensibility for product metadata

6.1.2 Backend Framework

Node.js with Express.js provides:

- Event-driven, non-blocking I/O for handling multiple concurrent requests
- RESTful API design for clean separation between frontend and backend
- Middleware support for authentication, validation, and error handling
- Connection pooling for efficient database access

6.1.3 Frontend Framework

React with Vite offers:

- Component-based architecture for reusable UI elements
- State management via Redux for complex application state
- Fast hot module replacement during development
- Optimized production builds with tree-shaking

6.1.4 Authentication

JWT (JSON Web Tokens) for stateless authentication:

- Secure token-based sessions
- Role-based access control (Customer, Shop, Admin)
- Token expiration and refresh mechanisms

7 Database Implementation

7.1 Account Management

7.1.1 ACCOUNT Table

The **ACCOUNT** table serves as the central authentication and identity store for all system users. It implements table-level inheritance, with specialized entities (CUSTOMER, SHOP, ADMIN) storing additional role-specific information.

Listing 1: ACCOUNT Table Creation

```
1 CREATE TABLE Account (  
2     account_id INT AUTO_INCREMENT PRIMARY KEY,  
3     email VARCHAR(255) UNIQUE NOT NULL,  
4     password VARCHAR(255) NOT NULL,  
5     role ENUM('Customer', 'Shop', 'Admin') NOT NULL,  
6     full_name VARCHAR(255) NOT NULL,  
7     phone VARCHAR(20) NOT NULL CHECK (LENGTH(phone) = 10 AND phone REGEXP  
8         '^[0-9]{10}$'),  
9     status ENUM('Active', 'Ban') DEFAULT 'Active',  
10    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
11    CONSTRAINT CK_AccountRole CHECK (role IN ('Customer', 'Shop', 'Admin'))  
);
```

General Description: The ACCOUNT table centralizes user authentication across all three user types. The **email** field is UNIQUE to prevent duplicate account registrations. The **phone** CHECK constraint enforces Vietnamese phone format validation at the database level. The **role** ENUM field determines user permissions (Customer can purchase, Shop can sell, Admin supervises). Status field enables account suspension without deletion.

7.1.2 CUSTOMER Table

Listing 2: CUSTOMER Table Creation

```
1 CREATE TABLE Customer (  
2     customer_id INT PRIMARY KEY,  
3     customer_code VARCHAR(20) UNIQUE,
```

```

4     address VARCHAR(255),
5     add_phone VARCHAR(20) CHECK (LENGTH(add_phone) = 10 AND add_phone REGEXP
      '^[0-9]{10}$'),
6     total_spent DECIMAL(15,2) DEFAULT 0,
7     total_order INT DEFAULT 0,
8     FOREIGN KEY (customer_id) REFERENCES Account(account_id) ON DELETE CASCADE
9 );

```

General Description: The CUSTOMER table extends ACCOUNT with purchase-specific attributes. The **customer_code** is auto-generated via trigger with 'CUST' prefix, enabling easy customer identification in reports. **Address** is mandatory for order fulfillment. **TotalSpent** and **TotalOrder** are derived attributes maintained by triggers on order completion, supporting customer loyalty analysis. The UNIQUE constraint on customer_code and index on total_spent enable efficient customer ranking and loyalty program implementation.

7.1.3 SHOP Table

Listing 3: SHOP Table Creation

```

1 CREATE TABLE Shop (
2     shop_id INT PRIMARY KEY,
3     shop_code VARCHAR(20) UNIQUE,
4     shop_name VARCHAR(255) NOT NULL,
5     shop_phone VARCHAR(20) CHECK (LENGTH(shop_phone) = 10 AND shop_phone REGEXP
      '^[0-9]{10}$'),
6     address_shop VARCHAR(255),
7     rating DECIMAL(3,2) DEFAULT 0 CHECK (rating BETWEEN 0 AND 5),
8     shop_status ENUM('Open', 'Temporarily Close', 'Closed') DEFAULT 'Open',
9     FOREIGN KEY (shop_id) REFERENCES Account(account_id) ON DELETE CASCADE
10 );

```

General Description: The SHOP table models seller entities with business-specific attributes. **shop_code** is auto-generated with 'SHOP' prefix for seller identification. **shop_status** controls whether a seller can list products (Open) or is inactive. **rating** is auto-calculated from customer reviews via trigger, providing trust metrics. Indexes on shop_code and rating enable efficient shop searching and ranking by customer ratings.

7.1.4 ADMIN Table

Listing 4: ADMIN Table Creation

```

1 CREATE TABLE Admin (
2     admin_id INT PRIMARY KEY,
3     role ENUM('Core', 'Moderator', 'Support') NOT NULL,
4     note TEXT,
5     FOREIGN KEY (admin_id) REFERENCES Account(account_id) ON DELETE CASCADE

```

6);

General Description: The ADMIN table represents platform supervisors with three access levels. **Core** admins have full system access, **Moderator** admins handle dispute resolution and content moderation, while **Support** admins assist customers. The note field allows tracking of admin activities and responsibilities. Unlike CUSTOMER and SHOP, ADMIN entities do not engage in transactions.

7.2 Product Catalog

7.2.1 CATEGORY Table

Listing 5: CATEGORY Table Creation

```

1 CREATE TABLE Category (
2     category_id INT AUTO_INCREMENT PRIMARY KEY,
3     category_name VARCHAR(100) NOT NULL,
4     parent_category_id INT,
5     FOREIGN KEY (parent_category_id) REFERENCES Category(category_id) ON DELETE SET
6     NULL
7 );

```

General Description: The CATEGORY table implements a self-referential recursive relationship enabling hierarchical product organization (e.g., Electronics → Phones → Smartphones). The **parent_category_id** can be NULL for root-level categories. The recursive FK with ON DELETE SET NULL ensures child categories remain accessible if a parent is deleted..

7.2.2 PRODUCT Table

Listing 6: PRODUCT Table Creation

```

1 CREATE TABLE Product (
2     product_id INT AUTO_INCREMENT PRIMARY KEY,
3     shop_id INT NOT NULL,
4     category_id INT NOT NULL,
5     product_name VARCHAR(255) NOT NULL,
6     description TEXT,
7     image VARCHAR(255),
8     rating DECIMAL(3,2) DEFAULT 0,
9     status ENUM('In stock', 'Out of stock') DEFAULT 'In stock',
10    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
11    FOREIGN KEY (shop_id) REFERENCES Shop(shop_id) ON DELETE CASCADE,
12    FOREIGN KEY (category_id) REFERENCES Category(category_id)
13 );

```

General Description: The PRODUCT table represents individual product listings managed by shops. Each product belongs to exactly one SHOP (N:1 relationship with ON DELETE CASCADE to remove products when shop is deleted) and one CATEGORY (N:1 with ON DELETE RESTRICT to preserve category structure). The **rating** is auto-calculated from customer reviews via trigger. **status** tracks inventory availability and is auto-updated by triggers when stock reaches zero. The updated_at timestamp enables tracking product modification history.

7.2.3 PRODUCTITEM Table

Listing 7: PRODUCTITEM Table Creation

```
1 CREATE TABLE ProductItem (  
2     item_id INT AUTO_INCREMENT PRIMARY KEY,  
3     product_id INT NOT NULL,  
4     shop_id INT NOT NULL,  
5     color VARCHAR(50) NOT NULL,  
6     type VARCHAR(50) NOT NULL,  
7     price DECIMAL(15,2) NOT NULL CHECK(price >= 0),  
8     stock INT NOT NULL CHECK(stock >= 0),  
9     image_url VARCHAR(255),  
10    FOREIGN KEY (product_id) REFERENCES Product(product_id) ON DELETE CASCADE,  
11    FOREIGN KEY (shop_id) REFERENCES Shop(shop_id) ON DELETE CASCADE  
12 );
```

General Description: The PRODUCTITEM (Variant) table represents purchasable variations of products. The UNIQUE constraint on (product_id, color, type) prevents duplicate variants of the same product. The **stock** field tracks inventory levels and is decremented by triggers on order placement, incremented on order cancellation. The **price** field is stored for each variant, enabling different pricing for sizes/colors.

7.3 Shopping Cart

7.3.1 CART Table

Listing 8: CART Table Creation

```
1 CREATE TABLE Cart (  
2     cart_id INT AUTO_INCREMENT PRIMARY KEY,  
3     customer_id INT UNIQUE NOT NULL,  
4     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
5     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
6     FOREIGN KEY (customer_id) REFERENCES Customer(customer_id) ON DELETE CASCADE  
7 );
```

General Description: The CART table implements a 1:1 relationship with CUSTOMER, ensuring each customer has exactly one shopping cart. The UNIQUE constraint on customer_id enforces this one-to-one relationship. The updated_at timestamp tracks cart modification time, useful for session management. ON DELETE CASCADE ensures cart deletion when customer account is removed.

7.3.2 CARTITEM Table

Listing 9: CARTITEM Table Creation

```
1 CREATE TABLE CartItem (  
2     cart_item_id INT AUTO_INCREMENT PRIMARY KEY,  
3     cart_id INT NOT NULL,  
4     item_id INT NOT NULL,  
5     quantity INT DEFAULT 1 CHECK(quantity >= 1),  
6     added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
7     FOREIGN KEY (cart_id) REFERENCES Cart(cart_id) ON DELETE CASCADE,  
8     FOREIGN KEY (item_id) REFERENCES ProductItem(item_id) ON DELETE CASCADE,  
9     UNIQUE KEY unique_cart_item (cart_id, item_id)  
10 );
```

General Description: The CARTITEM table implements the N:M relationship between CART and PRODUCTITEM. The UNIQUE constraint on (cart_id, item_id) prevents duplicate items in the same cart. The **quantity** field tracks the number of units selected, with CHECK constraint ensuring at least 1 unit. ON DELETE CASCADE ensures cart items are removed when either cart or product is deleted.

7.4 Orders and Items

7.4.1 ORDER Table

Listing 10: ORDER Table Creation

```
1 CREATE TABLE 'Order' (  
2     order_id INT AUTO_INCREMENT PRIMARY KEY,  
3     customer_id INT NOT NULL,  
4     shipping_id INT NOT NULL,  
5     voucher_id INT,  
6     status ENUM('Processing', 'Shipped', 'Delivered', 'Cancelled') DEFAULT  
7         'Processing',  
8     order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
9     delivered_date TIMESTAMP NULL,  
10    shipping_address VARCHAR(255) NOT NULL,  
11    total_amount DECIMAL(15,2) DEFAULT 0,  
12    payment_method ENUM('COD', 'Banking', 'Momo', 'ZaloPay') NOT NULL DEFAULT 'COD',  
13    note TEXT,  
14    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

14     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
15     FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
16     FOREIGN KEY (shipping_id) REFERENCES Shipping(shipping_id),
17     FOREIGN KEY (voucher_id) REFERENCES Voucher(voucher_id) ON DELETE SET NULL,
18     CONSTRAINT CK_OrderDateRange CHECK (delivered_date IS NULL OR delivered_date >=
19     order_date)

```

General Description: The ORDER table records all purchase transactions. The **status** ENUM tracks order lifecycle (Processing → Shipped → Delivered or Cancelled). The **voucher_id** is nullable (N:0..1 relationship) since orders may not use vouchers. The CHECK constraint **delivered_date >= order_date** enforces chronological consistency. Indexes on **customer_id**, **status**, and **order_date** enable efficient order retrieval and reporting. ON DELETE RESTRICT on **shipping_id** prevents accidental deletion of shipping methods in use.

7.4.2 ORDERITEM Table

Listing 11: ORDERITEM Table Creation

```

1 CREATE TABLE OrderItem (
2     order_item_id INT AUTO_INCREMENT PRIMARY KEY,
3     order_id INT NOT NULL,
4     item_id INT NOT NULL,
5     shop_id INT NOT NULL,
6     quantity INT NOT NULL CHECK(quantity >= 1),
7     price_at_purchase DECIMAL(15,2) NOT NULL CHECK(price_at_purchase >= 0),
8     FOREIGN KEY (order_id) REFERENCES 'Order'(order_id) ON DELETE CASCADE,
9     FOREIGN KEY (item_id) REFERENCES ProductItem(item_id),
10    FOREIGN KEY (shop_id) REFERENCES Shop(shop_id)
11 );

```

General Description: The ORDERITEM table represents individual line items within an order, supporting multi-seller purchases. The **price_at_purchase** field captures the price at transaction time, preserving historical accuracy even if product prices change. The **shop_id** field identifies the seller responsible for fulfilling each line item. The UNIQUE constraint on (order_id, item_id) prevents duplicate line items. ON DELETE RESTRICT on item_id and shop_id ensures order history remains intact.

7.5 Promotions

7.5.1 VOUCHER Table

Listing 12: VOUCHER Table Creation

```

1 CREATE TABLE Voucher (
2     voucher_id INT AUTO_INCREMENT PRIMARY KEY,

```



```

3   code VARCHAR(50) UNIQUE NOT NULL,
4   discount_type ENUM('Percentage', 'Amount') NOT NULL,
5   discount_value DECIMAL(10,2) NOT NULL CHECK(discount_value >= 0),
6   min_order_value DECIMAL(15,2) DEFAULT 0 CHECK(min_order_value >= 0),
7   start_date DATE NOT NULL,
8   expired_date DATE NOT NULL,
9   usage_limit INT DEFAULT 1 CHECK(usage_limit >= 1),
10  used_count INT DEFAULT 0,
11  status ENUM('Active', 'Expired') DEFAULT 'Active',
12  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
13  CONSTRAINT CK_VoucherDateRange CHECK (expired_date >= start_date)
14 );

```

General Description: The VOUCHER table manages promotional campaigns with flexible discount mechanisms. The **discount_type** ENUM supports both percentage (e.g., 10%) and fixed amount (e.g., 50,000 VND) discounts. The **min_order_value** enforces minimum purchase requirements. The **usage_limit** and **used_count** fields track voucher usage, preventing over-utilization. The CHECK constraint ensures **used_count** never exceeds **usage_limit**.

7.6 Shipping

7.6.1 SHIPPING Table

Listing 13: SHIPPING Table Creation

```

1 CREATE TABLE Shipping (
2   shipping_id INT AUTO_INCREMENT PRIMARY KEY,
3   name VARCHAR(100) NOT NULL,
4   estimated_days INT NOT NULL CHECK(estimated_days >= 0),
5   fee DECIMAL(10,2) NOT NULL CHECK(fee >= 0),
6   status ENUM('Active', 'Inactive') DEFAULT 'Active'
7 );

```

General Description: The SHIPPING table defines available delivery options for orders. Each record represents a logistics partner or delivery method. The **estimated_days** field provides expected delivery timeframes. The **fee** field enables dynamic shipping cost calculation. The **status** field controls availability (Active methods appear in checkout, Inactive ones are hidden).

7.7 Reviews and Feedback

7.7.1 REVIEW Table

Listing 14: REVIEW Table Creation

```

1 CREATE TABLE Review (
2   review_id INT AUTO_INCREMENT PRIMARY KEY,

```

```

3     customer_id INT NOT NULL,
4     target_type ENUM('Shop','Product') NOT NULL,
5     target_id INT NOT NULL,
6     rating INT NOT NULL CHECK(rating BETWEEN 1 AND 5),
7     comment TEXT NOT NULL,
8     image_url VARCHAR(255),
9     review_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
10    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id) ON DELETE CASCADE
11 );

```

8 Sample Data Insertion

This section demonstrates sample data for testing the database schema. Data is organized by entity relationships to maintain referential integrity.

8.1 Account Data

Listing 15: Sample ACCOUNT Insert

```

1  -- Insert 7 accounts: 3 customers, 3 shops, 1 admin
2  INSERT INTO Account (email, password, role, full_name, phone, status)
3  VALUES
4      ('c1@gmail.com', '1234', 'Customer', 'Nguyen Van A', '0901111111', 'Active'),
5      ('c2@gmail.com', '1234', 'Customer', 'Tran Thi B', '0902222222', 'Active'),
6      ('c3@gmail.com', '1234', 'Customer', 'Le Hoai C', '0903333333', 'Active'),
7      ('s1@gmail.com', '1234', 'Shop', 'Shop ABC', '0904444444', 'Active'),
8      ('s2@gmail.com', '1234', 'Shop', 'Shop XYZ', '0905555555', 'Active'),
9      ('s3@gmail.com', '1234', 'Shop', 'Tech Store', '0906666666', 'Active'),
10     ('admin1@gmail.com', '1234', 'Admin', 'Admin Core', '0907777777', 'Active');

```

8.2 Customer Data

Listing 16: Sample CUSTOMER Insert

```

1  -- Triggers automatically create CUSTOMER records during INSERT
2  -- This example shows updating customer info with addresses
3  UPDATE Customer SET
4      address = '123 Nguyen Hue, District 1, HCMC',
5      add_phone = '0908888888'
6  WHERE customer_id = 1;
7
8  UPDATE Customer SET
9      address = '456 Le Loi, District 3, HCMC',
10     add_phone = '0909999999'
11 WHERE customer_id = 2;
12

```

```

13 UPDATE Customer SET
14     address = '789 Tran Hung Dao, Hoan Kiem, Hanoi',
15     add_phone = '0911111111'
16 WHERE customer_id = 3;

```

8.3 Shop Data

Listing 17: Sample SHOP Insert

```

1  -- Triggers automatically create SHOP records
2  -- Update shop information
3  UPDATE Shop SET
4      shop_name = 'Shop ABC - Electronics',
5      shop_phone = '0904444444',
6      address_shop = '12 Vo Van Kiet, District 1',
7      rating = 4.50,
8      shop_status = 'Open'
9  WHERE shop_id = 4;
10
11 UPDATE Shop SET
12     shop_name = 'Shop XYZ - Fashion',
13     shop_phone = '0905555555',
14     address_shop = '45 Hai Ba Trung, District 3',
15     rating = 0,
16     shop_status = 'Open'
17 WHERE shop_id = 5;
18
19 UPDATE Shop SET
20     shop_name = 'Tech Store - Premium',
21     shop_phone = '0906666666',
22     address_shop = '67 Nguyen Dinh Chieu, District 7',
23     rating = 5.00,
24     shop_status = 'Open'
25 WHERE shop_id = 6;

```

8.4 Category Data

Listing 18: Sample CATEGORY Insert

```

1 INSERT INTO Category (category_name, parent_category_id)
2 VALUES
3     ('Electronics', NULL),          -- Root: ID 1
4     ('Fashion', NULL),             -- Root: ID 2
5     ('Phones', 1),                 -- Child of Electronics
6     ('Laptops', 1),                -- Child of Electronics
7     ('Clothes', 2),                -- Child of Fashion
8     ('Shoes', 2);                  -- Child of Fashion

```

8.5 Product and ProductItem Data

Listing 19: Sample PRODUCT Insert

```

1  -- Insert products from different shops
2  INSERT INTO Product (shop_id, category_id, product_name,
3                      description, image, status)
4  VALUES
5      (4, 3, 'iPhone 15 Pro',
6       'Latest Apple flagship with A17 Pro chip',
7       'iphone15pro.jpg', 'In stock'),
8
9      (4, 3, 'Samsung Galaxy S24',
10     'Latest Samsung with AI features',
11     'galaxyS24.jpg', 'In stock'),
12
13     (5, 5, 'Premium Cotton T-Shirt',
14     'High-quality 100% cotton shirt',
15     'tshirt.jpg', 'In stock'),
16
17     (6, 4, 'MacBook Air M3',
18     'Lightweight laptop for professionals',
19     'macbookm3.jpg', 'In stock'),
20
21     (6, 3, 'Sony Headset XM5',
22     'Noise-cancelling wireless headphones',
23     'headset.jpg', 'In stock');
```

Listing 20: Sample PRODUCTITEM Insert

```

1  -- Insert product variants (color, size, capacity)
2  INSERT INTO ProductItem (product_id, shop_id, color, type,
3                          price, stock, image_url)
4  VALUES
5      -- iPhone 15 Pro variants (Product 1, Shop 4)
6      (1, 4, 'Space Black', '128GB', 1099000, 50,
7       'iphone15pro-black-128.jpg'),
8
9      (1, 4, 'Blue Titanium', '256GB', 1199000, 30,
10     'iphone15pro-blue-256.jpg'),
11
12     -- Samsung Galaxy S24 variants (Product 2, Shop 4)
13     (2, 4, 'Phantom Black', '256GB', 920000, 40,
14     'galaxyS24-black-256.jpg'),
15
16     (2, 4, 'Amber Gold', '512GB', 1099000, 25,
17     'galaxyS24-gold-512.jpg'),
18
19     -- T-Shirt variants (Product 3, Shop 5)
20     (3, 5, 'White', 'Size M', 150000, 100,
```

```

21     'tshirt-white-m.jpg'),
22
23     (3, 5, 'Black', 'Size L', 150000, 80,
24     'tshirt-black-l.jpg'),
25
26     -- MacBook Air M3 variants (Product 4, Shop 6)
27     (4, 6, 'Silver', '16GB RAM', 2299000, 15,
28     'macbookm3-silver-16gb.jpg'),
29
30     (4, 6, 'Space Gray', '24GB RAM', 2599000, 10,
31     'macbookm3-gray-24gb.jpg'),
32
33     -- Sony Headset variants (Product 5, Shop 6)
34     (5, 6, 'Black', 'Standard', 4990000, 60,
35     'sonyxm5-black.jpg'),
36
37     (5, 6, 'Silver', 'Standard', 4990000, 40,
38     'sonyxm5-silver.jpg');

```

8.6 Shipping and Voucher Data

Listing 21: Sample SHIPPING Insert

```

1  INSERT INTO Shipping (name, estimated_days, fee, status)
2  VALUES
3      ('GHN Express', 2, 15000, 'Active'),
4      ('J&T Standard', 3, 12000, 'Active'),
5      ('Shopee Express', 1, 20000, 'Active'),
6      ('Ninja Van Economy', 4, 9000, 'Active');

```

Listing 22: Sample VOUCHER Insert

```

1  INSERT INTO Voucher (code, discount_type, discount_value,
2                      min_order_value, start_date, expired_date,
3                      usage_limit, status)
4  VALUES
5      -- 10% discount, min 500K
6      ('DISCOUNT10', 'Percentage', 10, 500000,
7      '2025-01-01', '2025-12-31', 1000, 'Active'),
8
9      -- Fixed 100K discount, min 1M
10     ('SAVE100K', 'Amount', 100000, 1000000,
11     '2025-06-01', '2025-12-31', 500, 'Active'),
12
13     -- New Year promotion: 5% off
14     ('NEWYEAR2025', 'Percentage', 5, 200000,
15     '2025-12-01', '2026-01-31', 2000, 'Active'),
16
17     -- Expired voucher (for testing)

```

```

18 ('OLDCODE', 'Percentage', 15, 300000,
19 '2024-01-01', '2024-12-31', 100, 'Expired');

```

8.7 Order Transaction Data

Listing 23: Sample ORDER Insert

```

1  -- Note: Cart and CartItem are auto-created by triggers
2  -- Orders created from shopping carts
3
4  -- Order 1: Customer 1, 2 items from Shop 4
5  INSERT INTO \Order\ (customer_id, shipping_id, voucher_id, status,
6                        shipping_address, total_amount,
7                        payment_method, note)
8  VALUES
9      (1, 1, 1, 'Delivered',
10       '123 Nguyen Hue, District 1, HCMC', 2098000,
11       'Banking', 'Received in good condition');
12
13 -- Insert order items for Order 1
14 INSERT INTO OrderItem (order_id, item_id, shop_id, quantity,
15                        price_at_purchase)
16 VALUES
17     (1, 1, 4, 1, 1099000), -- iPhone 15 Pro Black 128GB
18     (1, 2, 4, 1, 1199000); -- iPhone 15 Pro Blue 256GB
19
20 -- Order 2: Customer 2, cancelled order
21 INSERT INTO \Order\ (customer_id, shipping_id, voucher_id, status,
22                        shipping_address, total_amount,
23                        payment_method, note)
24 VALUES
25     (2, 2, NULL, 'Cancelled',
26      '456 Le Loi, District 3, HCMC', 932000,
27      'COD', 'Cancelled by customer');
28
29 INSERT INTO OrderItem (order_id, item_id, shop_id, quantity,
30                        price_at_purchase)
31 VALUES
32     (2, 3, 4, 1, 920000); -- Galaxy S24 Black 256GB
33
34 -- Order 3: Customer 3, delivered
35 INSERT INTO \Order\ (customer_id, shipping_id, voucher_id, status,
36                        shipping_address, total_amount,
37                        payment_method, note)
38 VALUES
39     (3, 3, 2, 'Delivered',
40      '789 Tran Hung Dao, Hanoi', 170000,
41      'Momo', 'Please leave at reception');
42

```

```

43 INSERT INTO OrderItem (order_id, item_id, shop_id, quantity,
44                        price_at_purchase)
45 VALUES
46     (3, 5, 5, 1, 150000); -- T-Shirt White Size M
47
48 -- Order 4: Customer 1, shipped
49 INSERT INTO \Order\ (customer_id, shipping_id, voucher_id, status,
50                     shipping_address, total_amount,
51                     payment_method, note)
52 VALUES
53     (1, 4, NULL, 'Shipped',
54      '123 Nguyen Hue, District 1, HCMC', 5009000,
55      'ZaloPay', NULL);
56
57 INSERT INTO OrderItem (order_id, item_id, shop_id, quantity,
58                        price_at_purchase)
59 VALUES
60     (4, 9, 6, 1, 4990000); -- Sony Headset Black

```

8.8 Review Data

Listing 24: Sample REVIEW Insert

```

1  -- Reviews only from verified purchases (Delivered orders)
2
3  -- Review 1: Customer 1 reviewed Product 1 (iPhone 15)
4  INSERT INTO Review (customer_id, target_type, target_id,
5                      rating, comment, image_url)
6  VALUES
7      (1, 'Product', 1, 5,
8       'Excellent phone! Fast performance and great camera',
9       'review_iphone.jpg'),
10
11  -- Review 2: Customer 1 reviewed Shop 4
12  (1, 'Shop', 4, 5,
13   'Very reliable seller, fast shipping',
14   NULL),
15
16  -- Review 3: Customer 3 reviewed Product 3 (T-Shirt)
17  (3, 'Product', 3, 4,
18   'Good quality cotton, true to size',
19   'review_tshirt.jpg'),
20
21  -- Review 4: Customer 3 reviewed Shop 5
22  (3, 'Shop', 5, 4,
23   'Decent shop, packaging could be better',
24   NULL),
25
26  -- Review 5: Customer 1 reviewed Product 5 (Headset)

```

```

27     (1, 'Product', 5, 5,
28         'Amazing sound quality, very comfortable',
29         'review_headset.jpg'),
30
31     -- Review 6: Customer 1 reviewed Shop 6
32     (1, 'Shop', 6, 5,
33         'Premium products, excellent customer service',
34         NULL);

```

9 Functions and Procedures

This section details the business logic automation implemented through MySQL stored functions, procedures, and triggers.

9.1 Account Management Functions

9.1.1 Function: Calculate Customer Total Spent

Purpose: Calculates the total amount spent by a customer across all non-cancelled orders, including shipping fees and voucher discounts.

Listing 25: Function: fn_get_customer_total_spent

```

1 CREATE FUNCTION fn_get_customer_total_spent(p_customer_id INT)
2 RETURNS DECIMAL(15, 2)
3 DETERMINISTIC
4 READS SQL DATA
5 BEGIN
6     DECLARE v_total_spent DECIMAL(15, 2);
7
8     SELECT COALESCE(SUM(fn_calculate_order_total(o.order_id)), 0)
9     INTO v_total_spent
10    FROM 'Order' o
11   WHERE o.customer_id = p_customer_id
12         AND o.status != 'Cancelled';
13
14     RETURN v_total_spent;
15 END$$

```

Related Tables: ACCOUNT, CUSTOMER, ORDER, ORDERITEM, SHIPPING, VOUCHER

Explanation: This function sums all completed order totals for a specific customer. It queries the ORDER table filtered by customer_id and status != 'Cancelled', then aggregates order totals including shipping fees and subtracting applied voucher discounts. The function is DETERMINISTIC (same input always returns same output) and reads SQL data only.

Example Usage:

```

1  -- Get total spent by customer ID 1
2  SELECT fn_get_customer_total_spent(1);
3  -- Result: 2,069,500.00 VND
4  -- Customer 1 orders: Order 1 (1,985,000) + Order 4 (84,500)
5
6  -- Get total spent by customer ID 2
7  SELECT fn_get_customer_total_spent(2);
8  -- Result: 0.00 VND
9  -- Customer 2 only has cancelled orders
10
11 -- Get total spent by customer ID 3
12 SELECT fn_get_customer_total_spent(3);
13 -- Result: 31,000.00 VND
14 -- Customer 3: Order 3 (31,000)
15
16 -- Get customer info with total spent
17 SELECT c.customer_code, a.full_name,
18        fn_get_customer_total_spent(c.customer_id) AS total_spent
19 FROM Customer c
20 INNER JOIN Account a ON c.customer_id = a.account_id
21 WHERE c.customer_id = 1;

```

9.2 Shop Management Functions

9.2.1 Function: Calculate Shop Revenue

Purpose: Calculates total revenue generated by a shop from all non-cancelled orders, accounting for multi-seller scenarios.

Listing 26: Function: fn_get_shop_revenue

```

1  CREATE FUNCTION fn_get_shop_revenue(p_shop_id INT)
2  RETURNS DECIMAL(15, 2)
3  DETERMINISTIC
4  READS SQL DATA
5  BEGIN
6      DECLARE v_revenue DECIMAL(15, 2);
7      DECLARE v_subtotal DECIMAL(15, 2);
8      DECLARE v_discount_total DECIMAL(15, 2);
9
10     -- Calculate shop revenue (OrderItem + Shipping - Voucher discount)
11     -- Only count non-cancelled orders
12
13     -- Calculate subtotal from OrderItem
14     SELECT COALESCE(SUM(oi.quantity * oi.price_at_purchase), 0)
15     INTO v_subtotal
16     FROM OrderItem oi
17     INNER JOIN 'Order' o ON oi.order_id = o.order_id

```

```

18 WHERE oi.shop_id = p_shop_id AND o.status != 'Cancelled';
19
20 -- Calculate voucher discount (only if min_order_value met)
21 SELECT COALESCE(SUM(
22     CASE
23         WHEN v.discount_type = 'Percentage' AND v_subtotal >=
24             COALESCE(v.min_order_value, 0)
25             THEN v_subtotal * v.discount_value / 100
26         WHEN v.discount_type = 'Amount' AND v_subtotal >=
27             COALESCE(v.min_order_value, 0)
28             THEN v.discount_value
29         ELSE 0
30     END
31 ), 0)
32 INTO v_discount_total
33 FROM 'Order' o
34 LEFT JOIN Voucher v ON o.voucher_id = v.voucher_id
35 WHERE o.order_id IN (
36     SELECT DISTINCT oi.order_id
37     FROM OrderItem oi
38     WHERE oi.shop_id = p_shop_id
39 )
40 AND o.status != 'Cancelled';
41
42 SET v_revenue = v_subtotal - v_discount_total;
43
44 IF v_revenue < 0 THEN
45     SET v_revenue = 0;
46 END IF;
47
48 RETURN v_revenue;
49 END$$

```

Related Tables: SHOP, ORDERITEM, ORDER

Explanation: This function calculates shop revenue by summing all OrderItem line items (quantity × price_at_purchase) where the shop_id matches and the order status is not 'Cancelled'. Since orders can contain items from multiple shops, using shop_id in OrderItem ensures accurate shop-specific revenue calculation.

Example Usage:

```

1 -- Get revenue for shop ID 4
2 SELECT fn_get_shop_revenue(4);
3 -- Result: 1,980,000.00 VND
4 -- Calculation: Shop 4 items (1,000,000 + 1,200,000) - voucher (220,000)
5
6 -- Get revenue for shop ID 5
7 SELECT fn_get_shop_revenue(5);

```

```

8  -- Result: 25,000.00 VND
9
10 -- Get revenue for shop ID 6
11 SELECT fn_get_shop_revenue(6);
12 -- Result: 80,000.00 VND
13
14 -- Top shops by revenue
15 SELECT s.shop_code, s.shop_name, fn_get_shop_revenue(s.shop_id) AS revenue
16 FROM Shop s
17 ORDER BY fn_get_shop_revenue(s.shop_id) DESC
18 LIMIT 5;

```

9.2.2 Function: Calculate Shop Rating

Purpose: Calculates the average rating of a shop based on verified customer reviews.

Listing 27: Function: fn_calculate_shop_rating

```

1  CREATE FUNCTION fn_calculate_shop_rating(p_shop_id INT)
2  RETURNS DECIMAL(3, 2)
3  DETERMINISTIC
4  READS SQL DATA
5  BEGIN
6      DECLARE v_avg_rating DECIMAL(3, 2);
7
8      SELECT COALESCE(AVG(r.rating), 0)
9      INTO v_avg_rating
10     FROM Review r
11     WHERE r.target_type = 'Shop'
12           AND r.target_id = p_shop_id;
13
14     RETURN v_avg_rating;
15 END$$

```

Related Tables: SHOP, REVIEW

Explanation: This function queries the REVIEW table where target_type='Shop' and target_id matches the shop_id, then calculates the average rating. If no reviews exist, it returns 0. The function is called by a trigger whenever a new review is added to keep the shop rating current.

Example Usage:

```

1  -- Get shop rating for shop 4
2  SELECT fn_calculate_shop_rating(4);
3  -- Result: 4.00
4  -- Reviews for Shop 4: (5 + 3) / 2 = 4.00
5

```

```

6  -- Get shop rating for shop 5
7  SELECT fn_calculate_shop_rating(5);
8  -- Result: 0.00
9  -- No reviews for Shop 5
10
11 -- Get shop rating for shop 6
12 SELECT fn_calculate_shop_rating(6);
13 -- Result: 5.00
14 -- Review for Shop 6: 5 stars
15
16 -- Get shop info with calculated rating
17 SELECT s.shop_code, s.shop_name, s.rating AS stored_rating,
18        fn_calculate_shop_rating(s.shop_id) AS calculated_rating
19 FROM Shop s
20 WHERE s.shop_id = 4;

```

9.3 Product Management Functions

9.3.1 Function: Calculate Product Rating

Purpose: Calculates the average rating of a product based on verified customer reviews.

Listing 28: Function: fn_calculate_product_rating

```

1  CREATE FUNCTION fn_calculate_product_rating(p_product_id INT)
2  RETURNS DECIMAL(3, 2)
3  DETERMINISTIC
4  READS SQL DATA
5  BEGIN
6      DECLARE v_avg_rating DECIMAL(3, 2);
7
8      SELECT COALESCE(AVG(r.rating), 0)
9      INTO v_avg_rating
10     FROM Review r
11     WHERE r.target_type = 'Product'
12           AND r.target_id = p_product_id;
13
14     RETURN v_avg_rating;
15 END$$

```

Related Tables: PRODUCT, REVIEW

Explanation: Similar to shop rating function, this calculates product average rating from REVIEW table where target_type='Product'. Returns 0 if no reviews exist. Used by triggers to maintain denormalized rating in PRODUCT table for query performance.

Example Usage:

```

1  -- Get rating for product 1
2  SELECT fn_calculate_product_rating(1);
3  -- Result: 4.50
4  -- Reviews for Product 1: (5 + 4) / 2 = 4.50
5
6  -- Get rating for product 3
7  SELECT fn_calculate_product_rating(3);
8  -- Result: 4.00
9  -- Review for Product 3: 4 stars
10
11 -- Get rating for product 4
12 SELECT fn_calculate_product_rating(4);
13 -- Result: 5.00
14 -- Review for Product 4: 5 stars
15
16 -- Get top-rated products
17 SELECT p.product_id, p.product_name,
18        fn_calculate_product_rating(p.product_id) AS rating,
19        COUNT(r.review_id) AS review_count
20 FROM Product p
21 LEFT JOIN Review r ON p.product_id = r.target_id
22                    AND r.target_type = 'Product'
23 GROUP BY p.product_id
24 ORDER BY rating DESC
25 LIMIT 10;

```

9.3.2 Function: Calculate Order Total

Purpose: Calculates the final order total including subtotal, shipping fee, and voucher discount with minimum order value validation.

Listing 29: Function: fn_calculate_order_total

```

1  CREATE FUNCTION fn_calculate_order_total(p_order_id INT)
2  RETURNS DECIMAL(15, 2)
3  DETERMINISTIC
4  READS SQL DATA
5  BEGIN
6      DECLARE v_subtotal DECIMAL(15, 2);
7      DECLARE v_shipping_fee DECIMAL(10, 2);
8      DECLARE v_discount DECIMAL(10, 2);
9      DECLARE v_discount_type VARCHAR(20);
10     DECLARE v_discount_value DECIMAL(10, 2);
11     DECLARE v_min_order_value DECIMAL(10, 2);
12     DECLARE v_total DECIMAL(15, 2);
13
14     -- Calculate subtotal from order items
15     SELECT COALESCE(SUM(oi.quantity * oi.price_at_purchase), 0)
16     INTO v_subtotal

```

```

17     FROM OrderItem oi
18     WHERE oi.order_id = p_order_id;
19
20     -- Get shipping fee
21     SELECT COALESCE(sh.fee, 0)
22     INTO v_shipping_fee
23     FROM 'Order' o
24     LEFT JOIN Shipping sh ON o.shipping_id = sh.shipping_id
25     WHERE o.order_id = p_order_id;
26
27     -- Get voucher discount with min_order_value check
28     SELECT v.discount_type, COALESCE(v.discount_value, 0),
29           COALESCE(v.min_order_value, 0)
30     INTO v_discount_type, v_discount_value, v_min_order_value
31     FROM 'Order' o
32     LEFT JOIN Voucher v ON o.voucher_id = v.voucher_id
33     WHERE o.order_id = p_order_id;
34
35     -- Calculate discount amount (only if subtotal meets minimum requirement)
36     SET v_discount = 0;
37     IF v_discount_type IS NOT NULL AND v_subtotal >= v_min_order_value THEN
38         IF v_discount_type = 'Percentage' THEN
39             SET v_discount = v_subtotal * (v_discount_value / 100);
40         ELSEIF v_discount_type = 'Amount' THEN
41             SET v_discount = v_discount_value;
42         END IF;
43     END IF;
44
45     -- Calculate total = subtotal + shipping - discount
46     SET v_total = v_subtotal + v_shipping_fee - v_discount;
47
48     RETURN v_total;
49 END$$

```

Related Tables: ORDER, ORDERITEM, SHIPPING, VOUCHER

Explanation: This comprehensive function calculates order totals by:

1. Summing all OrderItem line items (quantity × price_at_purchase)
2. Adding shipping fee from related Shipping record
3. Calculating voucher discount (percentage or fixed amount) with min_order_value validation
4. Subtracting discount from subtotal + shipping
5. Ensuring final total is non-negative

Example Usage:

```

1  -- Calculate order total for order 1
2  SELECT fn_calculate_order_total(1);
3  -- Result: 1,985,000.00 VND
4  -- Calculation: (1,000,000 + 1,200,000) + 5,000 - 220,000
5
6  -- Calculate order total for order 2
7  SELECT fn_calculate_order_total(2);
8  -- Result: 904,000.00 VND
9  -- Calculation: 900,000 + 4,000
10
11 -- Get order summary with calculated total
12 SELECT o.order_id, o.order_date, a.full_name,
13        fn_calculate_order_total(o.order_id) AS total_amount,
14        o.status, o.payment_method
15 FROM 'Order' o
16 INNER JOIN Customer c ON o.customer_id = c.customer_id
17 INNER JOIN Account a ON c.customer_id = a.account_id
18 WHERE o.order_id = 1;

```

9.3.3 Function: Get Order Total Items with Cursor

Purpose: Demonstrates cursor usage to loop through OrderItems and calculate total quantity in an order.

Listing 30: Function: fn_get_order_total_items_with_cursor

```

1  CREATE FUNCTION fn_get_order_total_items_with_cursor(p_order_id INT)
2  RETURNS INT
3  DETERMINISTIC
4  READS SQL DATA
5  BEGIN
6      DECLARE v_total_items INT DEFAULT 0;
7      DECLARE v_done INT DEFAULT FALSE;
8      DECLARE v_item_quantity INT;
9
10     -- Declare cursor for OrderItems
11     DECLARE order_items_cursor CURSOR FOR
12         SELECT quantity
13         FROM OrderItem
14         WHERE order_id = p_order_id;
15
16     -- Declare continue handler for cursor
17     DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;
18
19     -- Validate order exists
20     IF NOT EXISTS (SELECT 1 FROM 'Order' WHERE order_id = p_order_id) THEN
21         RETURN 0;
22     END IF;
23

```

```
24  -- Open cursor
25  OPEN order_items_cursor;
26
27  -- Loop through each item
28  item_loop: LOOP
29      FETCH order_items_cursor INTO v_item_quantity;
30
31      IF v_done THEN
32          LEAVE item_loop;
33      END IF;
34
35      SET v_total_items = v_total_items + v_item_quantity;
36  END LOOP item_loop;
37
38  -- Close cursor
39  CLOSE order_items_cursor;
40
41  RETURN v_total_items;
42 END$$
43 DELIMITER ;
```

Related Tables: ORDER, ORDERITEM

Explanation: This function demonstrates advanced cursor usage in MySQL stored functions. It:

1. Declares a cursor to iterate through all OrderItems for a specific order
2. Uses CONTINUE HANDLER to catch NOT FOUND exception when cursor reaches end
3. Loops through each item, accumulating total quantity
4. Returns cumulative total items count

Example Usage:

```
1  -- Get total items in order 1
2  SELECT fn_get_order_total_items_with_cursor(1);
3  -- Result: 2
4  -- Order 1 has 2 items (qty 1 + qty 1 = 2 total units)
5
6  -- Get total items in order 2
7  SELECT fn_get_order_total_items_with_cursor(2);
8  -- Result: 1
9  -- Order 2 has 1 item (qty 1)
10
11 -- Get total items for non-existent order
12 SELECT fn_get_order_total_items_with_cursor(999);
13 -- Result: 0
14 -- Order 999 does not exist
```



```

15
16 -- Order summary with total items
17 SELECT o.order_id,
18        COUNT(oi.order_item_id) AS line_items,
19        fn_get_order_total_items_with_cursor(o.order_id) AS total_items
20 FROM 'Order' o
21 LEFT JOIN OrderItem oi ON o.order_id = oi.order_id
22 WHERE o.order_id = 1
23 GROUP BY o.order_id;

```

9.4 Category Hierarchy Procedures

9.4.1 Procedure: Get Category Hierarchy (All Levels)

Purpose: Displays complete hierarchical structure of all categories using recursive CTE (Common Table Expression).

Listing 31: Procedure: sp_get_category_hierarchy

```

1 CREATE PROCEDURE sp_get_category_hierarchy()
2 BEGIN
3     WITH RECURSIVE category_tree AS (
4         -- Anchor: Get all root categories (parent_category_id IS NULL)
5         SELECT
6             category_id,
7             category_name,
8             parent_category_id,
9             0 AS level,
10            CAST(category_name AS CHAR(500)) AS path
11        FROM Category
12        WHERE parent_category_id IS NULL
13
14        UNION ALL
15
16        -- Recursive: Get all child categories
17        SELECT
18            c.category_id,
19            c.category_name,
20            c.parent_category_id,
21            ct.level + 1,
22            CONCAT(ct.path, ' > ', c.category_name)
23        FROM Category c
24        INNER JOIN category_tree ct ON c.parent_category_id = ct.category_id
25        WHERE ct.level < 10 -- Prevent infinite loop, limit to 10 levels
26    )
27    SELECT
28        category_id,
29        category_name,
30        parent_category_id,

```

```

31     level,
32     path,
33     REPEAT(' ', level) AS indent
34 FROM category_tree
35 ORDER BY path;
36 END$$

```

Related Tables: CATEGORY

Explanation: This procedure uses MySQL's recursive CTE (WITH RECURSIVE) to traverse the category hierarchy:

1. **Anchor Query:** Selects all root categories (parent_category_id IS NULL) at level 0
2. **Recursive Query:** Joins child categories to the tree, incrementing level
3. **Path Building:** Constructs a breadcrumb path (e.g., "Electronics > Phones > Smartphones")
4. **Recursion Limit:** Stops at level 10 to prevent infinite loops

Example Usage:

```

1  -- Get complete category hierarchy
2  CALL sp_get_category_hierarchy();
3
4  -- Result:
5  -- category_id | category_name | parent_id | level | path
6  -- 1           | Electronics  | NULL     | 0     | Electronics
7  -- 3           | Phones      | 1        | 1     | Electronics > Phones
8  -- 4           | Laptops     | 1        | 1     | Electronics > Laptops
9  -- 2           | Fashion     | NULL     | 0     | Fashion
10 -- 5           | Clothes     | 2        | 1     | Fashion > Clothes

```

9.4.2 Procedure: Get All Children of a Category

Purpose: Retrieves all descendant categories (children, grandchildren, etc.) of a specified parent category.

Listing 32: Procedure: sp_get_category_all_children

```

1 CREATE PROCEDURE sp_get_category_all_children(IN p_category_id INT)
2 BEGIN
3     IF NOT EXISTS (SELECT 1 FROM Category WHERE category_id = p_category_id) THEN
4         SIGNAL SQLSTATE '45000'
5         SET MESSAGE_TEXT = 'Category does not exist';
6     END IF;
7
8     WITH RECURSIVE children_tree AS (

```

```

9      -- Anchor: Get the root category
10     SELECT
11         category_id,
12         category_name,
13         parent_category_id,
14         0 AS level
15     FROM Category
16     WHERE category_id = p_category_id
17
18     UNION ALL
19
20     -- Recursive: Get all child categories at next level
21     SELECT
22         c.category_id,
23         c.category_name,
24         c.parent_category_id,
25         ct.level + 1
26     FROM Category c
27     INNER JOIN children_tree ct ON c.parent_category_id = ct.category_id
28     WHERE ct.level < 10
29 )
30 SELECT
31     category_id,
32     category_name,
33     parent_category_id,
34     level,
35     REPEAT(' ', level) AS indent
36 FROM children_tree
37 ORDER BY level, category_name;
38 END$$

```

Related Tables: CATEGORY

Explanation: This procedure finds all descendants of a category:

1. Validates that the input category exists (raises error if not)
2. Anchor query retrieves the starting category at level 0
3. Recursive query finds all immediate children, then their children, etc.
4. Returns hierarchically structured results with indentation

Example Usage:

```

1  -- Get all children of Electronics (category_id = 1)
2  CALL sp_get_category_all_children(1);
3
4  -- Result:
5  -- category_id | category_name | parent_id | level

```

```

6  -- 1          | Electronics | NULL      | 0
7  -- 3          | Phones    | 1         | 1
8  -- 4          | Laptops   | 1         | 1
9
10 -- Get all children of Fashion (category_id = 2)
11 CALL sp_get_category_all_children(2);
12
13 -- Result:
14 -- category_id | category_name | parent_id | level
15 -- 2          | Fashion      | NULL      | 0
16 -- 5          | Clothes     | 2         | 1

```

9.4.3 Procedure: Get All Parents of a Category

Purpose: Retrieves all ancestor categories (parent, grandparent, etc.) of a specified category, showing the reverse hierarchy path.

Listing 33: Procedure: sp_get_category_all_parents

```

1  CREATE PROCEDURE sp_get_category_all_parents(IN p_category_id INT)
2  BEGIN
3      IF NOT EXISTS (SELECT 1 FROM Category WHERE category_id = p_category_id) THEN
4          SIGNAL SQLSTATE '45000'
5          SET MESSAGE_TEXT = 'Category does not exist';
6      END IF;
7
8      WITH RECURSIVE parents_tree AS (
9          -- Anchor: Get the root category
10         SELECT
11             category_id,
12             category_name,
13             parent_category_id,
14             0 AS level
15         FROM Category
16         WHERE category_id = p_category_id
17
18         UNION ALL
19
20         -- Recursive: Get all parent categories at upper level
21         SELECT
22             c.category_id,
23             c.category_name,
24             c.parent_category_id,
25             pt.level + 1
26         FROM Category c
27         INNER JOIN parents_tree pt ON pt.parent_category_id = c.category_id
28         WHERE pt.level < 10
29     )
30     SELECT

```

```

31     category_id,
32     category_name,
33     parent_category_id,
34     level,
35     REPEAT(' ', level) AS indent
36 FROM parents_tree
37 ORDER BY level DESC, category_name;
38 END$$

```

Related Tables: CATEGORY

Explanation: This procedure traverses the hierarchy upward to find all ancestors:

1. Validates category existence
2. Anchor retrieves the specified category
3. Recursive query follows parent_category_id chain upward
4. Returns results ordered from leaf to root (DESC level)

Example Usage:

```

1  -- Get all parents of Phones (category_id = 3)
2  CALL sp_get_category_all_parents(3);
3
4  -- Result (showing breadcrumb trail):
5  -- category_id | category_name | parent_id | level
6  -- 3          | Phones       | 1         | 0
7  -- 1          | Electronics  | NULL      | 1
8  -- This shows: Phones < Electronics
9
10 -- Get all parents of Clothes (category_id = 5)
11 CALL sp_get_category_all_parents(5);
12
13 -- Result:
14 -- category_id | category_name | parent_id | level
15 -- 5          | Clothes       | 2         | 0
16 -- 2          | Fashion      | NULL      | 1
17 -- This shows: Clothes < Fashion

```

9.5 Order Management Procedures

9.5.1 Procedure: Create Order from Cart

Purpose: Complex business logic procedure that creates an order from customer's shopping cart, validates inventory, applies vouchers, and updates related tables.

Listing 34: Procedure: sp_create_order_from_cart (Part 1)

```
1 CREATE PROCEDURE sp_create_order_from_cart(  
2     IN p_customer_id INT,  
3     IN p_shipping_id INT,  
4     IN p_voucher_id INT,  
5     IN p_shipping_address VARCHAR(255),  
6     IN p_payment_method VARCHAR(50),  
7     IN p_note TEXT,  
8     OUT p_order_id INT  
9 )  
10 BEGIN  
11     DECLARE v_subtotal DECIMAL(15, 2) DEFAULT 0;  
12     DECLARE v_shipping_fee DECIMAL(10, 2) DEFAULT 0;  
13     DECLARE v_discount DECIMAL(10, 2) DEFAULT 0;  
14     DECLARE v_total_amount DECIMAL(15, 2) DEFAULT 0;  
15     DECLARE v_cart_id INT;  
16     DECLARE v_cart_count INT DEFAULT 0;  
17     DECLARE v_discount_type VARCHAR(20);  
18     DECLARE v_discount_value DECIMAL(10, 2);  
19     DECLARE v_min_order_value DECIMAL(10, 2);  
20     DECLARE v_usage_limit INT;  
21     DECLARE v_used_count INT;  
22  
23     -- Validation checks  
24     IF NOT EXISTS (SELECT 1 FROM Customer WHERE customer_id = p_customer_id) THEN  
25         SIGNAL SQLSTATE '45000'  
26         SET MESSAGE_TEXT = 'Customer does not exist';  
27     END IF;  
28  
29     IF NOT EXISTS (SELECT 1 FROM Shipping WHERE shipping_id = p_shipping_id  
30         AND status = 'Active') THEN  
31         SIGNAL SQLSTATE '45000'  
32         SET MESSAGE_TEXT = 'Invalid or inactive shipping method';  
33     END IF;  
34  
35     -- Get and validate cart  
36     SELECT cart_id INTO v_cart_id  
37     FROM Cart  
38     WHERE customer_id = p_customer_id;  
39  
40     IF v_cart_id IS NULL THEN  
41         SIGNAL SQLSTATE '45000'  
42         SET MESSAGE_TEXT = 'Cart not found';  
43     END IF;  
44  
45     SELECT COUNT(*) INTO v_cart_count  
46     FROM CartItem  
47     WHERE cart_id = v_cart_id;  
48  
49     IF v_cart_count = 0 THEN
```

```

50     SIGNAL SQLSTATE '45000'
51     SET MESSAGE_TEXT = 'Cart is empty';
52 END IF;
53
54 -- Calculate order amounts
55 SELECT COALESCE(SUM(ci.quantity * pi.price), 0)
56 INTO v_subtotal
57 FROM CartItem ci
58 INNER JOIN ProductItem pi ON ci.item_id = pi.item_id
59 WHERE ci.cart_id = v_cart_id;
60
61 SELECT fee INTO v_shipping_fee
62 FROM Shipping
63 WHERE shipping_id = p_shipping_id;

```

Listing 35: Procedure: sp_create_order_from_cart (Part 2)

```

1
2 -- Apply voucher if provided
3 IF p_voucher_id IS NOT NULL THEN
4     SELECT discount_type, discount_value, min_order_value,
5           usage_limit, used_count
6     INTO v_discount_type, v_discount_value, v_min_order_value,
7           v_usage_limit, v_used_count
8     FROM Voucher
9     WHERE voucher_id = p_voucher_id
10        AND status = 'Active'
11        AND expired_date > CURDATE();
12
13 IF v_discount_type IS NULL THEN
14     SIGNAL SQLSTATE '45000'
15     SET MESSAGE_TEXT = 'Voucher is invalid or expired';
16 END IF;
17
18 IF v_subtotal < v_min_order_value THEN
19     SIGNAL SQLSTATE '45000'
20     SET MESSAGE_TEXT = 'Order value does not meet voucher minimum';
21 END IF;
22
23 IF v_usage_limit > 0 AND v_used_count >= v_usage_limit THEN
24     SIGNAL SQLSTATE '45000'
25     SET MESSAGE_TEXT = 'Voucher usage limit reached';
26 END IF;
27
28 -- Calculate discount
29 IF v_discount_type = 'Percentage' THEN
30     SET v_discount = v_subtotal * (v_discount_value / 100);
31 ELSEIF v_discount_type = 'Amount' THEN
32     SET v_discount = v_discount_value;
33 END IF;

```

```

34
35     -- Update voucher usage
36     UPDATE Voucher
37     SET used_count = used_count + 1
38     WHERE voucher_id = p_voucher_id;
39 END IF;
40
41     -- Calculate total
42     SET v_total_amount = v_subtotal + v_shipping_fee - v_discount;
43     IF v_total_amount < 0 THEN
44         SET v_total_amount = 0;
45     END IF;
46
47     -- Create order
48     INSERT INTO 'Order' (
49         customer_id, shipping_id, voucher_id, status,
50         shipping_address, total_amount, payment_method, note
51     ) VALUES (
52         p_customer_id, p_shipping_id, p_voucher_id, 'Processing',
53         p_shipping_address, v_total_amount, p_payment_method, p_note
54     );
55
56     SET p_order_id = LAST_INSERT_ID();
57
58     -- Move cart items to order items
59     INSERT INTO OrderItem (order_id, item_id, shop_id, quantity, price_at_purchase)
60     SELECT p_order_id, ci.item_id, pi.shop_id, ci.quantity, pi.price
61     FROM CartItem ci
62     INNER JOIN ProductItem pi ON ci.item_id = pi.item_id
63     WHERE ci.cart_id = v_cart_id;
64
65     -- Clear cart
66     DELETE FROM CartItem WHERE cart_id = v_cart_id;
67
68     -- Update customer stats
69     UPDATE Customer
70     SET total_order = total_order + 1,
71         total_spent = total_spent + v_total_amount
72     WHERE customer_id = p_customer_id;
73 END$$

```

Related Tables: CUSTOMER, CART, CARTITEM, ORDER, ORDERITEM, SHIPPING, VOUCHER, PRODUCTITEM

Explanation: This is a complex transaction procedure that:

1. Validates customer, shipping method, and cart existence
2. Checks that cart contains items

3. Calculates subtotal from cart items
4. Validates and applies voucher (with min order value check)
5. Creates ORDER record with final total
6. Converts CARTITEM records to ORDERITEM records
7. Clears the shopping cart
8. Updates customer statistics (total_order, total_spent)

Example Usage:

```

1  -- Setup: Add items to Customer 1's cart
2  DELETE FROM CartItem WHERE cart_id = 1;
3  INSERT INTO CartItem (cart_id, item_id, quantity) VALUES (1, 1, 1);
4  INSERT INTO CartItem (cart_id, item_id, quantity) VALUES (1, 4, 2);
5
6  -- Verify cart contents before order creation
7  SELECT ci.cart_id, ci.item_id, ci.quantity, pi.price
8  FROM CartItem ci
9  INNER JOIN ProductItem pi ON ci.item_id = pi.item_id
10 WHERE ci.cart_id = 1;
11 -- Result: 2 rows (item 1, item 4)
12
13 -- Create order from customer 1's cart
14 CALL sp_create_order_from_cart(
15     1,                -- customer_id
16     1,                -- shipping_id
17     NULL,             -- voucher_id (no voucher)
18     '123 Updated Street', -- shipping_address
19     'Banking',        -- payment_method
20     'Test from cart', -- note
21     @new_order_id     -- OUT parameter
22 );
23
24 SELECT @new_order_id AS order_id;
25 -- Result: New order_id (e.g., 5)
26
27 -- Verify order was created
28 SELECT order_id, customer_id, status, payment_method
29 FROM \Order\ WHERE order_id = @new_order_id;
30 -- Result: status='Processing', payment_method='Banking'
31
32 -- Verify cart is empty after order creation
33 SELECT COUNT(*) AS remaining_items FROM CartItem WHERE cart_id = 1;
34 -- Result: 0 (cart cleared after order creation)

```

9.5.2 Procedure: Apply Voucher

Purpose: Validates a voucher code and calculates discount amount with comprehensive business rule checking.

Listing 36: Procedure: sp_apply_voucher

```

1 CREATE PROCEDURE sp_apply_voucher(
2     IN p_voucher_code VARCHAR(50),
3     IN p_order_amount DECIMAL(15, 2),
4     OUT p_discount_amount DECIMAL(10, 2),
5     OUT p_is_valid TINYINT,
6     OUT p_message VARCHAR(255)
7 )
8 BEGIN
9     DECLARE v_voucher_id INT;
10    DECLARE v_discount_type VARCHAR(20);
11    DECLARE v_discount_value DECIMAL(10, 2);
12    DECLARE v_min_order_value DECIMAL(10, 2);
13    DECLARE v_usage_limit INT;
14    DECLARE v_used_count INT;
15    DECLARE v_status VARCHAR(20);
16
17    SET p_is_valid = 0;
18    SET p_discount_amount = 0;
19
20    -- Get voucher by code
21    SELECT
22        voucher_id, discount_type, discount_value, min_order_value,
23        usage_limit, used_count, status
24    INTO
25        v_voucher_id, v_discount_type, v_discount_value, v_min_order_value,
26        v_usage_limit, v_used_count, v_status
27    FROM Voucher
28    WHERE code = p_voucher_code;
29
30    -- Validation chain
31    IF v_voucher_id IS NULL THEN
32        SET p_message = 'Voucher code not found';
33    ELSEIF v_status != 'Active' THEN
34        SET p_message = 'Voucher is not active';
35    ELSEIF CURDATE() > (SELECT expired_date FROM Voucher WHERE voucher_id =
36        v_voucher_id) THEN
37        SET p_message = 'Voucher has expired';
38    ELSEIF v_usage_limit > 0 AND v_used_count >= v_usage_limit THEN
39        SET p_message = 'Voucher usage limit reached';
40    ELSEIF p_order_amount < v_min_order_value THEN
41        SET p_message = CONCAT('Minimum order value is ', v_min_order_value);
42    ELSE
43        -- Calculate discount
44        CASE v_discount_type

```

```

44         WHEN 'Percentage' THEN
45             SET p_discount_amount = p_order_amount * (v_discount_value / 100);
46         WHEN 'Amount' THEN
47             SET p_discount_amount = v_discount_value;
48     END CASE;
49
50     -- Cap discount at order amount
51     IF p_discount_amount > p_order_amount THEN
52         SET p_discount_amount = p_order_amount;
53     END IF;
54
55     SET p_is_valid = 1;
56     SET p_message = 'Voucher applied successfully';
57 END IF;
58 END$$

```

Related Tables: VOUCHER

Explanation: This procedure validates vouchers through a series of checks:

1. Checks if voucher code exists in database
2. Verifies voucher is in 'Active' status
3. Checks expiration date against current date
4. Validates usage count hasn't exceeded limit
5. Validates order amount meets minimum requirement
6. Calculates discount (percentage or fixed amount)
7. Caps discount to not exceed order amount

Example Usage:

```

1  -- Test 1: Valid percentage voucher (DISCOUNT10)
2  SET @discount = 0;
3  SET @valid = FALSE;
4  SET @msg = '';
5  CALL sp_apply_voucher('DISCOUNT10', 500000, @discount, @valid, @msg);
6  SELECT @discount AS discount_amount, @valid AS is_valid, @msg AS message;
7  -- Result: discount=50000, is_valid=1, message='Voucher applied successfully'
8  -- DISCOUNT10 is 10% percentage discount
9
10 -- Test 2: Valid amount voucher (SAVE50) - Order meets minimum
11 CALL sp_apply_voucher('SAVE50', 300000, @discount, @valid, @msg);
12 SELECT @discount, @valid, @msg;
13 -- Result: discount=50000, is_valid=1, message='Voucher applied successfully'
14

```

```

15 -- Test 3: Order below minimum requirement
16 CALL sp_apply_voucher('SAVE50', 150000, @discount, @valid, @msg);
17 SELECT @msg;
18 -- Result: message mentions minimum order value not met
19
20 -- Test 4: Non-existent voucher
21 CALL sp_apply_voucher('NONEXISTENT', 500000, @discount, @valid, @msg);
22 SELECT @msg;
23 -- Result: 'Voucher code not found'

```

9.6 Shop Management Procedures

9.6.1 Procedure: Get Shop Revenue Report

Purpose: Comprehensive reporting procedure that generates shop revenue summary and top-selling products for a date range.

Listing 37: Procedure: sp_get_shop_revenue_report

```

1 CREATE PROCEDURE sp_get_shop_revenue_report(
2     IN p_shop_id INT,
3     IN p_from_date DATE,
4     IN p_to_date DATE
5 )
6 BEGIN
7     -- Validate shop exists
8     IF NOT EXISTS (SELECT 1 FROM Shop WHERE shop_id = p_shop_id) THEN
9         SIGNAL SQLSTATE '45000'
10        SET MESSAGE_TEXT = 'Shop does not exist';
11    END IF;
12
13    -- Main revenue summary
14    SELECT
15        s.shop_id,
16        s.shop_code,
17        s.shop_name,
18        COUNT(DISTINCT o.order_id) AS total_orders,
19        COUNT(DISTINCT oi.order_item_id) AS total_line_items,
20        SUM(oi.quantity) AS total_units_sold,
21        COALESCE(SUM(oi.quantity * oi.price_at_purchase), 0) AS total_revenue,
22        COALESCE(AVG(oi.quantity * oi.price_at_purchase), 0) AS avg_order_value,
23        s.rating AS shop_rating,
24        p_from_date AS report_from,
25        p_to_date AS report_to
26    FROM Shop s
27    LEFT JOIN OrderItem oi ON s.shop_id = oi.shop_id
28    LEFT JOIN 'Order' o ON oi.order_id = o.order_id
29        AND o.order_date BETWEEN p_from_date AND DATE_ADD(p_to_date, INTERVAL 1 DAY)
30        AND o.status != 'Cancelled'

```

```

31 WHERE s.shop_id = p_shop_id
32 GROUP BY s.shop_id, s.shop_code, s.shop_name, s.rating;
33
34 -- Top 10 selling product items
35 SELECT
36     p.product_id,
37     p.product_name,
38     pi.color,
39     pi.type,
40     COUNT(oi.order_item_id) AS times_ordered,
41     SUM(oi.quantity) AS total_quantity_sold,
42     SUM(oi.quantity * oi.price_at_purchase) AS item_revenue,
43     ROUND(SUM(oi.quantity * oi.price_at_purchase) / COUNT(oi.order_item_id), 2)
44     AS avg_item_value
45 FROM Product p
46 INNER JOIN ProductItem pi ON p.product_id = pi.product_id
47 INNER JOIN OrderItem oi ON pi.item_id = oi.item_id
48 INNER JOIN 'Order' o ON oi.order_id = o.order_id
49     AND o.order_date BETWEEN p_from_date AND DATE_ADD(p_to_date, INTERVAL 1 DAY)
50     AND o.status != 'Cancelled'
51 WHERE pi.shop_id = p_shop_id
52 GROUP BY p.product_id, p.product_name, pi.color, pi.type
53 ORDER BY item_revenue DESC
54 LIMIT 10;
55 END$$

```

Related Tables: SHOP, ORDER, ORDERITEM, PRODUCT, PRODUCTITEM

Explanation: This reporting procedure generates two result sets:

Result Set 1 - Shop Summary:

- Total orders during period
- Line items and units sold
- Revenue metrics (total and average)
- Current shop rating
- Date range of report

Result Set 2 - Top 10 Products:

- Product and variant details (color, type)
- Order frequency
- Revenue contribution

- Average item value

Example Usage:

```

1  -- Get revenue report for shop 4 for full year 2025
2  CALL sp_get_shop_revenue_report(4, '2025-01-01', '2025-12-31');
3
4  -- Result Set 1 (Summary):
5  -- shop_id | shop_code | shop_name | total_orders | total_units | revenue
6  -- 4       | SHOP00004 | Shop ABC | 1             | 2           | 2,200,000
7
8  -- Result Set 2 (Top Products):
9  -- product_name | color | type | times_ordered | qty_sold | revenue
10 -- iPhone 15    | Red   | 128GB | 1             | 1        | 1,000,000
11 -- iPhone 15    | Blue  | 256GB | 1             | 1        | 1,200,000
12
13 -- Get revenue report for shop 5
14 CALL sp_get_shop_revenue_report(5, '2025-01-01', '2025-12-31');
15
16 -- Result Set 1:
17 -- shop_id | shop_code | shop_name | total_orders | total_units | revenue
18 -- 5       | SHOP00005 | Shop XYZ | 1             | 1          | 25,000
19
20 -- Result Set 2:
21 -- product_name | color | type | times_ordered | qty_sold | revenue
22 -- T-Shirt      | White | Size M | 1             | 1        | 25,000

```

9.7 Product CRUD Procedures

9.7.1 Procedure: Add Product

Purpose: Creates a new product in the system with validation for shop and category existence.

Listing 38: Procedure: sp_add_product

```

1  CREATE PROCEDURE sp_add_product(
2      IN p_shop_id INT,
3      IN p_category_id INT,
4      IN p_product_name VARCHAR(255),
5      IN p_description TEXT,
6      IN p_image VARCHAR(255),
7      OUT p_product_id INT,
8      OUT p_status VARCHAR(50)
9  )
10 BEGIN
11     DECLARE EXIT HANDLER FOR SQLEXCEPTION
12     BEGIN
13         SET p_product_id = -1;
14         SET p_status = 'Error: Invalid shop or category ID';
15     END;

```

```

16
17 SET p_product_id = -1;
18 SET p_status = 'Error';
19
20 -- Validate shop exists
21 IF NOT EXISTS (SELECT 1 FROM Shop WHERE shop_id = p_shop_id) THEN
22     SET p_status = 'Error: Shop not found';
23 ELSEIF NOT EXISTS (SELECT 1 FROM Category
24                     WHERE category_id = p_category_id) THEN
25     SET p_status = 'Error: Category not found';
26 ELSE
27     -- Insert product
28     INSERT INTO Product (shop_id, category_id, product_name,
29                          description, image, status)
30     VALUES (p_shop_id, p_category_id, p_product_name,
31             p_description, p_image, 'In stock');
32
33     SET p_product_id = LAST_INSERT_ID();
34     SET p_status = 'Success';
35 END IF;
36 END$$

```

Related Tables: PRODUCT, SHOP, CATEGORY

Explanation: This procedure creates a new product with error handling:

1. **EXIT HANDLER:** Catches SQL exceptions and returns error status
2. **Shop Validation:** Verifies shop_id exists in Shop table
3. **Category Validation:** Verifies category_id exists in Category table
4. **Product Creation:** Inserts new product with default status 'In stock'
5. **OUT Parameters:** Returns new product_id and status message

Example Usage:

```

1 -- Add new product to Shop 4, Category 3 (Phones)
2 CALL sp_add_product(
3     4,                -- shop_id
4     3,                -- category_id
5     'Samsung Galaxy S24', -- product_name
6     'Latest Samsung flagship', -- description
7     'samsung-s24.jpg',   -- image
8     @product_id,         -- OUT: product_id
9     @status              -- OUT: status message
10 );
11
12 SELECT @product_id, @status;

```

```

13 -- Result: @product_id = 5, @status = 'Success'
14
15 -- Verify product was created
16 SELECT product_id, product_name, status
17 FROM Product WHERE product_id = @product_id;
18 -- Result: Samsung Galaxy S24, In stock
19
20 -- Test invalid shop ID
21 CALL sp_add_product(
22     999,                                -- invalid shop_id
23     3,
24     'Test Product',
25     'Description',
26     'test.jpg',
27     @product_id,
28     @status
29 );
30
31 SELECT @status;
32 -- Result: 'Error: Shop not found'

```

9.7.2 Procedure: Update Product

Purpose: Updates existing product information with NULL-safe field updates.

Listing 39: Procedure: sp_update_product

```

1 CREATE PROCEDURE sp_update_product(
2     IN p_product_id INT,
3     IN p_product_name VARCHAR(255),
4     IN p_description TEXT,
5     IN p_image VARCHAR(255),
6     IN p_status VARCHAR(50),
7     OUT p_success BOOLEAN,
8     OUT p_message VARCHAR(100)
9 )
10 BEGIN
11     DECLARE EXIT HANDLER FOR SQLEXCEPTION
12     BEGIN
13         SET p_success = FALSE;
14         SET p_message = 'Error: Database error';
15     END;
16
17     IF NOT EXISTS (SELECT 1 FROM Product WHERE product_id = p_product_id) THEN
18         SET p_success = FALSE;
19         SET p_message = 'Error: Product not found';
20     ELSE
21         UPDATE Product
22         SET product_name = COALESCE(p_product_name, product_name),
23             description = COALESCE(p_description, description),

```



```

24         image = COALESCE(p_image, image),
25         status = COALESCE(p_status, status)
26     WHERE product_id = p_product_id;
27
28     SET p_success = TRUE;
29     SET p_message = 'Product updated successfully';
30 END IF;
31 END$$

```

Related Tables: PRODUCT

Explanation: This procedure demonstrates selective field updates:

1. **COALESCE Pattern:** Updates only non-NULL input fields
2. **Existence Check:** Validates product_id before update
3. **Error Handling:** Returns success flag and descriptive message
4. **Partial Updates:** Allows updating individual fields without affecting others

Example Usage:

```

1  -- Update only product name and status
2  CALL sp_update_product(
3      1,                                -- product_id
4      'iPhone 15 Pro Max',              -- new name
5      NULL,                             -- keep existing description
6      NULL,                             -- keep existing image
7      'Out of stock',                   -- new status
8      @success,
9      @message
10 );
11
12 SELECT @success, @message;
13 -- Result: TRUE, 'Product updated successfully'
14
15 -- Verify update
16 SELECT product_name, status FROM Product WHERE product_id = 1;
17 -- Result: iPhone 15 Pro Max, Out of stock
18
19 -- Update only description
20 CALL sp_update_product(
21     1,
22     NULL,                             -- keep existing name
23     'Updated description text',        -- new description
24     NULL,
25     NULL,
26     @success,
27     @message

```

28);

9.7.3 Procedure: Delete Product

Purpose: Deletes a product and all related product variants with CASCADE behavior.

Listing 40: Procedure: sp_delete_product

```

1 CREATE PROCEDURE sp_delete_product(
2     IN p_product_id INT,
3     OUT p_success BOOLEAN,
4     OUT p_message VARCHAR(100)
5 )
6 BEGIN
7     DECLARE v_affected_items INT;
8
9     DECLARE EXIT HANDLER FOR SQLEXCEPTION
10 BEGIN
11     SET p_success = FALSE;
12     SET p_message = 'Error: Database error';
13 END;
14
15 IF NOT EXISTS (SELECT 1 FROM Product WHERE product_id = p_product_id) THEN
16     SET p_success = FALSE;
17     SET p_message = 'Error: Product not found';
18 ELSE
19     -- Get count of affected items before deletion
20     SELECT COUNT(*) INTO v_affected_items
21     FROM ProductItem
22     WHERE product_id = p_product_id;
23
24     -- Delete product (CASCADE deletes ProductItems)
25     DELETE FROM Product WHERE product_id = p_product_id;
26
27     SET p_success = TRUE;
28     SET p_message = CONCAT('Product deleted. Removed ',
29                             v_affected_items, ' variant(s)');
30 END IF;
31 END$$

```

Related Tables: PRODUCT, PRODUCTITEM

Explanation: This procedure handles cascading deletion:

1. **Pre-deletion Count:** Records number of variants before deletion
2. **CASCADE Behavior:** Foreign key constraints automatically delete related ProductItems

3. **Informative Message:** Returns count of deleted variants
4. **Transactional Safety:** EXIT HANDLER ensures atomicity

Example Usage:

```

1  -- Check product variants before deletion
2  SELECT COUNT(*) AS variant_count
3  FROM ProductItem
4  WHERE product_id = 1;
5  -- Result: 2 variants (Red 128GB, Blue 256GB)
6
7  -- Delete product
8  CALL sp_delete_product(1, @success, @message);
9
10 SELECT @success, @message;
11 -- Result: TRUE, 'Product deleted. Removed 2 variant(s)'
12
13 -- Verify deletion
14 SELECT COUNT(*) FROM Product WHERE product_id = 1;
15 -- Result: 0
16
17 SELECT COUNT(*) FROM ProductItem WHERE product_id = 1;
18 -- Result: 0 (cascaded deletion)

```

9.7.4 Procedure: Add Product Item (Variant)

Purpose: Adds a new product variant with color, type, price, and stock information.

Listing 41: Procedure: sp_add_product_item

```

1  CREATE PROCEDURE sp_add_product_item(
2      IN p_product_id INT,
3      IN p_shop_id INT,
4      IN p_color VARCHAR(50),
5      IN p_type VARCHAR(50),
6      IN p_price DECIMAL(15,2),
7      IN p_stock INT,
8      IN p_image_url VARCHAR(255),
9      OUT p_item_id INT,
10     OUT p_status VARCHAR(50)
11 )
12 BEGIN
13     DECLARE EXIT HANDLER FOR SQLEXCEPTION
14     BEGIN
15         SET p_item_id = -1;
16         SET p_status = 'Error: Invalid input';
17     END;
18
19     SET p_item_id = -1;
20     SET p_status = 'Error';

```

```

21
22 IF NOT EXISTS (SELECT 1 FROM Product WHERE product_id = p_product_id) THEN
23     SET p_status = 'Error: Product not found';
24 ELSEIF p_price <= 0 THEN
25     SET p_status = 'Error: Price must be > 0';
26 ELSEIF p_stock < 0 THEN
27     SET p_status = 'Error: Stock cannot be negative';
28 ELSE
29     INSERT INTO ProductItem (product_id, shop_id, color, type,
30                             price, stock, image_url)
31     VALUES (p_product_id, p_shop_id, p_color, p_type,
32             p_price, p_stock, p_image_url);
33
34     SET p_item_id = LAST_INSERT_ID();
35     SET p_status = 'Success';
36 END IF;
37 END$$

```

Related Tables: PRODUCTITEM, PRODUCT, SHOP

Explanation: This procedure creates product variants with validation:

1. **Product Existence:** Validates parent product exists
2. **Price Validation:** Ensures price > 0
3. **Stock Validation:** Ensures stock >= 0
4. **Variant Details:** Stores color, type (size/capacity), image

Example Usage:

```

1  -- Add new variant: iPhone 15 Green 512GB
2  CALL sp_add_product_item(
3      1,                -- product_id (iPhone 15)
4      4,                -- shop_id
5      'Green',          -- color
6      '512GB',          -- type
7      1400000.00,       -- price
8      15,               -- stock
9      'iphone15-green-512.jpg', -- image_url
10     @item_id,
11     @status
12 );
13
14 SELECT @item_id, @status;
15 -- Result: @item_id = 5, @status = 'Success'
16
17 -- Verify variant was created
18 SELECT item_id, color, type, price, stock

```

```

19 FROM ProductItem WHERE item_id = @item_id;
20 -- Result: 5, Green, 512GB, 1400000.00, 15
21
22 -- Test invalid price
23 CALL sp_add_product_item(
24     1, 4, 'Black', '128GB',
25     -100.00, -- invalid price
26     10, 'test.jpg',
27     @item_id, @status
28 );
29
30 SELECT @status;
31 -- Result: 'Error: Price must be > 0'

```

9.7.5 Procedure: Update Product Item

Purpose: Updates product variant price, stock, and image with validation.

Listing 42: Procedure: sp_update_product_item

```

1 CREATE PROCEDURE sp_update_product_item(
2     IN p_item_id INT,
3     IN p_price DECIMAL(15,2),
4     IN p_stock INT,
5     IN p_image_url VARCHAR(255),
6     OUT p_success BOOLEAN,
7     OUT p_message VARCHAR(100)
8 )
9 BEGIN
10     DECLARE EXIT HANDLER FOR SQLEXCEPTION
11     BEGIN
12         SET p_success = FALSE;
13         SET p_message = 'Error: Database error';
14     END;
15
16     IF NOT EXISTS (SELECT 1 FROM ProductItem WHERE item_id = p_item_id) THEN
17         SET p_success = FALSE;
18         SET p_message = 'Error: Item not found';
19     ELSEIF p_price IS NOT NULL AND p_price <= 0 THEN
20         SET p_success = FALSE;
21         SET p_message = 'Error: Price must be > 0';
22     ELSEIF p_stock IS NOT NULL AND p_stock < 0 THEN
23         SET p_success = FALSE;
24         SET p_message = 'Error: Stock cannot be negative';
25     ELSE
26         UPDATE ProductItem
27         SET price = COALESCE(p_price, price),
28             stock = COALESCE(p_stock, stock),
29             image_url = COALESCE(p_image_url, image_url)
30         WHERE item_id = p_item_id;

```

```

31
32     SET p_success = TRUE;
33     SET p_message = 'Item updated successfully';
34 END IF;
35 END$$

```

Related Tables: PRODUCTITEM

Explanation: This procedure demonstrates selective updates with validation:

1. **COALESCE Updates:** Updates only provided fields
2. **Pre-update Validation:** Checks constraints before UPDATE
3. **Business Rules:** Price must be positive, stock non-negative

Example Usage:

```

1  -- Update only price and stock
2  CALL sp_update_product_item(
3      1,                      -- item_id
4      1100000.00,             -- new price
5      50,                     -- new stock
6      NULL,                   -- keep existing image
7      @success,
8      @message
9  );
10
11 SELECT @success, @message;
12 -- Result: TRUE, 'Item updated successfully'
13
14 -- Update only stock
15 CALL sp_update_product_item(
16     1,
17     NULL,                      -- keep existing price
18     45,                        -- decrease stock
19     NULL,
20     @success,
21     @message
22 );

```

9.7.6 Procedure: Delete Product Item

Purpose: Deletes a specific product variant.

Listing 43: Procedure: sp_delete_product_item

```

1 CREATE PROCEDURE sp_delete_product_item(
2     IN p_item_id INT,

```

```

3      OUT p_success BOOLEAN,
4      OUT p_message VARCHAR(100)
5  )
6  BEGIN
7      DECLARE EXIT HANDLER FOR SQLEXCEPTION
8      BEGIN
9          SET p_success = FALSE;
10         SET p_message = 'Error: Database error';
11     END;
12
13     IF NOT EXISTS (SELECT 1 FROM ProductItem WHERE item_id = p_item_id) THEN
14         SET p_success = FALSE;
15         SET p_message = 'Error: Item not found';
16     ELSE
17         DELETE FROM ProductItem WHERE item_id = p_item_id;
18
19         SET p_success = TRUE;
20         SET p_message = 'Item deleted successfully';
21     END IF;
22 END$$

```

Related Tables: PRODUCTITEM

Explanation: Simple deletion procedure with existence validation and error handling.

Example Usage:

```

1  -- Delete variant
2  CALL sp_delete_product_item(2, @success, @message);
3
4  SELECT @success, @message;
5  -- Result: TRUE, 'Item deleted successfully'
6
7  -- Verify deletion
8  SELECT COUNT(*) FROM ProductItem WHERE item_id = 2;
9  -- Result: 0

```

9.8 Product Search Procedure

9.8.1 Procedure: Get Product List with Filters

Purpose: Dynamic product search with filters, sorting, aggregations, and pagination using dynamic SQL.

Listing 44: Procedure: sp_get_product_list (Excerpt)

```

1  CREATE PROCEDURE sp_get_product_list(
2      IN p_search VARCHAR(255),

```

```

3      IN p_category_id INT,
4      IN p_shop_id INT,
5      IN p_min_price DECIMAL(15,2),
6      IN p_max_price DECIMAL(15,2),
7      IN p_status VARCHAR(50),
8      IN p_sort_by VARCHAR(50),
9      IN p_sort_order VARCHAR(10),
10     IN p_page INT,
11     IN p_limit INT,
12     OUT p_total_count INT
13 )
14 BEGIN
15     DECLARE v_offset INT;
16     DECLARE v_order_clause VARCHAR(100);
17
18     SET v_offset = (p_page - 1) * p_limit;
19
20     -- Build ORDER clause
21     SET v_order_clause = CASE p_sort_by
22         WHEN 'price' THEN CONCAT('min_price ', COALESCE(p_sort_order, 'ASC'))
23         WHEN 'rating' THEN CONCAT('avg_rating ', COALESCE(p_sort_order, 'DESC'))
24         WHEN 'created_at' THEN CONCAT('p.created_at ', COALESCE(p_sort_order,
25             'DESC'))
26         ELSE 'p.created_at DESC'
27     END;
28
29     -- Count total matching records
30     SELECT COUNT(DISTINCT p.product_id) INTO p_total_count
31     FROM Product p
32     INNER JOIN Shop s ON p.shop_id = s.shop_id
33     INNER JOIN Category c ON p.category_id = c.category_id
34     LEFT JOIN ProductItem pi ON p.product_id = pi.product_id
35     WHERE 1=1
36         AND (p_search IS NULL OR p.product_name LIKE CONCAT('%', p_search, '%')
37             OR p.description LIKE CONCAT('%', p_search, '%'))
38         AND (p_category_id IS NULL OR p.category_id = p_category_id)
39         AND (p_shop_id IS NULL OR p.shop_id = p_shop_id)
40         AND (p_status IS NULL OR p.status = p_status);
41
42     -- Dynamic query construction
43     SET @sql = CONCAT('
44         SELECT
45             p.product_id,
46             p.product_name,
47             p.description,
48             p.image,
49             p.status,
50             p.created_at,
51             p.shop_id,
52             s.shop_name,

```



```

52         s.rating as shop_rating,
53         c.category_id,
54         c.category_name,
55         MIN(pi.price) as min_price,
56         MAX(pi.price) as max_price,
57         SUM(pi.stock) as total_stock,
58         COUNT(DISTINCT pi.item_id) as variant_count,
59         COALESCE((
60             SELECT AVG(r.rating)
61             FROM Review r
62             WHERE r.target_id = p.product_id AND r.target_type = "Product"
63         ), 0) as avg_rating,
64         COALESCE((
65             SELECT COUNT(*)
66             FROM Review r
67             WHERE r.target_id = p.product_id AND r.target_type = "Product"
68         ), 0) as review_count
69     FROM Product p
70     INNER JOIN Shop s ON p.shop_id = s.shop_id
71     INNER JOIN Category c ON p.category_id = c.category_id
72     LEFT JOIN ProductItem pi ON p.product_id = pi.product_id
73     WHERE 1=1',
74     CASE WHEN p_search IS NOT NULL THEN
75         CONCAT(' AND (p.product_name LIKE "%', p_search, '%" OR p.description
76             LIKE "%', p_search, '%"'))
77     ELSE '' END,
78     CASE WHEN p_category_id IS NOT NULL THEN CONCAT(' AND p.category_id = ',
79         p_category_id)
80     ELSE '' END,
81     CASE WHEN p_shop_id IS NOT NULL THEN CONCAT(' AND p.shop_id = ', p_shop_id)
82     ELSE '' END,
83     CASE WHEN p_status IS NOT NULL THEN CONCAT(' AND p.status = "', p_status,
84         '"')
85     ELSE '' END,
86     ' GROUP BY p.product_id, p.product_name, p.description, p.image, p.status,
87         p.created_at, p.shop_id, s.shop_name, s.rating, c.category_id,
88         c.category_name',
89     CASE WHEN p_min_price IS NOT NULL THEN CONCAT(' HAVING min_price >= ',
90         p_min_price)
91     ELSE '' END,
92     CASE WHEN p_max_price IS NOT NULL THEN
93         CONCAT(CASE WHEN p_min_price IS NOT NULL THEN ' AND' ELSE ' HAVING' END,
94             ' max_price <= ', p_max_price)
95     ELSE '' END,
96     ' ORDER BY ', v_order_clause,
97     ' LIMIT ', p_limit, ' OFFSET ', v_offset
98 );
99
100 PREPARE stmt FROM @sql;
101 EXECUTE stmt;

```

```

97  DEALLOCATE PREPARE stmt;
98  END$$

```

Related Tables: PRODUCT, PRODUCTITEM, SHOP, CATEGORY, REVIEW

Explanation: This advanced procedure demonstrates:

1. **Dynamic SQL Construction:** Builds query based on provided filters
2. **Multiple Filters:** Search text, category, shop, status, price range
3. **Aggregations:** MIN/MAX price, SUM stock, COUNT variants, AVG rating
4. **Subqueries:** Rating and review count calculated via subselects
5. **Sorting:** Dynamic sort order based on price/rating/date
6. **Pagination:** Offset and limit for result set paging
7. **Total Count:** OUT parameter returns total matching records

Example Usage:

```

1  -- Search smartphones under 10M, sorted by rating, page 1
2  DECLARE @total INT;
3  CALL sp_get_product_list(
4      'iPhone',      -- search term
5      3,             -- category_id (Smartphones)
6      NULL,          -- shop_id (any)
7      0,             -- min_price
8      10000000,      -- max_price
9      'In stock',    -- status
10     'rating',       -- sort by rating
11     'DESC',         -- descending
12     1,             -- page 1
13     20,            -- 20 items per page
14     @total         -- OUT: total count
15 );
16
17 SELECT @total; -- e.g., 45 total matches

```

10 Database Triggers

This section documents the automated database triggers that enforce business rules, maintain data integrity, and handle derived attributes.

10.1 Order Management Triggers

10.1.1 Trigger: Order Item Before Insert

Purpose: Validates stock availability and sets derived values (price_at_purchase, shop_id) before inserting order items.

Listing 45: Trigger: trg_order_item_before_insert

```
1 CREATE TRIGGER trg_order_item_before_insert
2 BEFORE INSERT ON OrderItem
3 FOR EACH ROW
4 BEGIN
5     DECLARE v_item_price DECIMAL(10, 2);
6     DECLARE v_item_stock INT;
7     DECLARE v_shop_id INT;
8
9     -- Get product item details
10    SELECT price, stock, shop_id
11    INTO v_item_price, v_item_stock, v_shop_id
12    FROM ProductItem
13    WHERE item_id = NEW.item_id;
14
15    -- Validate item exists
16    IF v_item_price IS NULL THEN
17        SIGNAL SQLSTATE '45000'
18        SET MESSAGE_TEXT = 'Product item does not exist';
19    END IF;
20
21    -- Check stock availability
22    IF v_item_stock < NEW.quantity THEN
23        SIGNAL SQLSTATE '45000'
24        SET MESSAGE_TEXT = 'Insufficient stock for this product item';
25    END IF;
26
27    -- Set price at purchase if not provided
28    IF NEW.price_at_purchase IS NULL OR NEW.price_at_purchase = 0 THEN
29        SET NEW.price_at_purchase = v_item_price;
30    END IF;
31
32    -- Set shop_id if not provided
33    IF NEW.shop_id IS NULL OR NEW.shop_id = 0 THEN
34        SET NEW.shop_id = v_shop_id;
35    END IF;
36 END$$
```

Trigger Type: BEFORE INSERT

Related Tables: ORDERITEM, PRODUCTITEM

Business Logic: This trigger enforces critical order integrity:

1. **Existence Validation:** Ensures item_id references valid ProductItem
2. **Stock Validation:** Prevents ordering more items than available stock
3. **Price Capture:** Records current price as price_at_purchase for historical accuracy
4. **Shop Association:** Automatically links order item to correct shop

Example Scenario:

```

1  -- Scenario 1: Valid order item insertion
2  -- ProductItem 1: price=1,000,000, stock=10
3  INSERT INTO OrderItem (order_id, item_id, quantity)
4  VALUES (1, 1, 2);
5  -- Success: price_at_purchase=1,000,000, shop_id set automatically
6
7  -- Scenario 2: Insufficient stock
8  -- ProductItem 2: stock=5
9  INSERT INTO OrderItem (order_id, item_id, quantity)
10 VALUES (1, 2, 10);
11 -- Error: 'Insufficient stock for this product item'
12
13 -- Scenario 3: Invalid item_id
14 INSERT INTO OrderItem (order_id, item_id, quantity)
15 VALUES (1, 999, 1);
16 -- Error: 'Product item does not exist'

```

10.1.2 Trigger: Order Item After Insert

Purpose: Decreases product stock after order item insertion and updates product status if out of stock.

Listing 46: Trigger: trg_order_item_after_insert

```

1  CREATE TRIGGER trg_order_item_after_insert
2  AFTER INSERT ON OrderItem
3  FOR EACH ROW
4  BEGIN
5      DECLARE v_product_id INT;
6      DECLARE v_total_stock INT;
7
8      -- Decrease product item stock
9      UPDATE ProductItem
10 SET stock = stock - NEW.quantity
11 WHERE item_id = NEW.item_id;
12
13 -- Get product_id and check total stock
14 SELECT product_id INTO v_product_id
15 FROM ProductItem

```

```

16     WHERE item_id = NEW.item_id;
17
18     SELECT SUM(stock) INTO v_total_stock
19     FROM ProductItem
20     WHERE product_id = v_product_id;
21
22     -- Update product status if all variants are out of stock
23     IF v_total_stock <= 0 THEN
24         UPDATE Product
25         SET status = 'Out of stock'
26         WHERE product_id = v_product_id;
27     END IF;
28 END$$

```

Trigger Type: AFTER INSERT

Related Tables: ORDERITEM, PRODUCTITEM, PRODUCT

Business Logic: Maintains inventory consistency:

1. **Stock Deduction:** Automatically decreases ProductItem stock by ordered quantity
2. **Aggregate Check:** Calculates total stock across all product variants
3. **Status Update:** Marks product as 'Out of stock' when all variants depleted

Example Scenario:

```

1  -- Before: iPhone 15 has 2 variants
2  -- Variant 1 (Red): stock=10
3  -- Variant 2 (Blue): stock=5
4
5  -- Customer orders 10 Red iPhones
6  INSERT INTO OrderItem (order_id, item_id, quantity)
7  VALUES (1, 1, 10);
8
9  -- After trigger execution:
10 -- Variant 1 (Red): stock=0
11 -- Variant 2 (Blue): stock=5
12 -- Product status: Still 'Active' (total stock = 5)
13
14 -- Customer orders 5 Blue iPhones
15 INSERT INTO OrderItem (order_id, item_id, quantity)
16 VALUES (1, 2, 5);
17
18 -- After trigger execution:
19 -- Variant 1 (Red): stock=0
20 -- Variant 2 (Blue): stock=0
21 -- Product status: 'Out of stock' (total stock = 0)

```

10.1.3 Trigger: Order After Update (Stock Restoration)

Purpose: Restores product stock when an order is cancelled, using cursor to iterate through order items.

Listing 47: Trigger: trg_order_after_update

```
1 CREATE TRIGGER trg_order_after_update
2 AFTER UPDATE ON 'Order'
3 FOR EACH ROW
4 BEGIN
5     DECLARE v_product_id INT;
6     DECLARE v_total_stock INT;
7
8     -- If order status changed from non-Cancelled to Cancelled
9     IF OLD.status != 'Cancelled' AND NEW.status = 'Cancelled' THEN
10         BEGIN
11             DECLARE done INT DEFAULT FALSE;
12             DECLARE v_item_id INT;
13             DECLARE v_quantity INT;
14
15             DECLARE item_cursor CURSOR FOR
16                 SELECT item_id, quantity
17                 FROM OrderItem
18                 WHERE order_id = NEW.order_id;
19
20             DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
21
22             OPEN item_cursor;
23
24             restore_loop: LOOP
25                 FETCH item_cursor INTO v_item_id, v_quantity;
26                 IF done THEN
27                     LEAVE restore_loop;
28                 END IF;
29
30                 -- Restore stock for this item
31                 UPDATE ProductItem
32                 SET stock = stock + v_quantity
33                 WHERE item_id = v_item_id;
34
35                 -- Get product_id
36                 SELECT product_id INTO v_product_id
37                 FROM ProductItem
38                 WHERE item_id = v_item_id;
39
40                 -- Check total stock for this product
41                 SELECT SUM(stock) INTO v_total_stock
42                 FROM ProductItem
43                 WHERE product_id = v_product_id;
44
```

```

45      -- Update product status back to Active if stock available
46      IF v_total_stock > 0 THEN
47          UPDATE Product
48          SET status = 'Active'
49          WHERE product_id = v_product_id
50              AND status = 'Out of stock';
51      END IF;
52  END LOOP;
53
54      CLOSE item_cursor;
55  END;
56  END IF;
57  END$$

```

Trigger Type: AFTER UPDATE

Related Tables: ORDER, ORDERITEM, PRODUCTITEM, PRODUCT

Business Logic: Handles order cancellation with cursor-based iteration:

1. **Status Change Detection:** Only executes when order status changes to 'Cancelled'
2. **Cursor Loop:** Iterates through all OrderItems in the cancelled order
3. **Stock Restoration:** Adds back ordered quantity to ProductItem stock
4. **Status Reactivation:** Changes Product status from 'Out of stock' to 'Active'

Example Scenario:

```

1  -- Order 1 contains:
2  -- Item 1 (Red iPhone): quantity=10
3  -- Item 2 (Blue iPhone): quantity=5
4
5  -- Before cancellation:
6  -- Variant 1: stock=0
7  -- Variant 2: stock=0
8  -- Product status: 'Out of stock'
9
10 -- Cancel the order
11 UPDATE 'Order' SET status = 'Cancelled' WHERE order_id = 1;
12
13 -- After trigger execution:
14 -- Variant 1: stock=10 (restored)
15 -- Variant 2: stock=5 (restored)
16 -- Product status: 'Active' (reactivated)

```

10.2 Review Management Triggers

10.2.1 Trigger: Review Before Insert (Verified Purchase)

Purpose: Enforces verified purchase requirement - customers can only review products/shops from delivered orders.

Listing 48: Trigger: trg_review_before_insert

```

1 CREATE TRIGGER trg_review_before_insert
2 BEFORE INSERT ON Review
3 FOR EACH ROW
4 BEGIN
5     -- Enforce rating between 1 and 5
6     IF NEW.rating < 1 OR NEW.rating > 5 THEN
7         SIGNAL SQLSTATE '45000'
8         SET MESSAGE_TEXT = 'Rating must be between 1 and 5';
9     END IF;
10
11    -- Validate based on target type
12    IF NEW.target_type = 'Product' THEN
13        -- Check if customer purchased this product
14        IF NOT EXISTS (
15            SELECT 1
16            FROM OrderItem oi
17            INNER JOIN 'Order' o ON oi.order_id = o.order_id
18            INNER JOIN ProductItem pi ON oi.item_id = pi.item_id
19            WHERE pi.product_id = NEW.target_id
20                  AND o.customer_id = NEW.customer_id
21                  AND o.status = 'Delivered'
22        ) THEN
23            SIGNAL SQLSTATE '45000'
24            SET MESSAGE_TEXT =
25                'Can only review products from delivered orders';
26        END IF;
27    ELSEIF NEW.target_type = 'Shop' THEN
28        -- Check if customer purchased from this shop
29        IF NOT EXISTS (
30            SELECT 1
31            FROM OrderItem oi
32            INNER JOIN 'Order' o ON oi.order_id = o.order_id
33            WHERE oi.shop_id = NEW.target_id
34                  AND o.customer_id = NEW.customer_id
35                  AND o.status = 'Delivered'
36        ) THEN
37            SIGNAL SQLSTATE '45000'
38            SET MESSAGE_TEXT =
39                'Can only review shops you have purchased from';
40        END IF;
41    END IF;
42 END$$

```


Trigger Type: BEFORE INSERT

Related Tables: REVIEW, ORDER, ORDERITEM, PRODUCTITEM

Business Logic: Implements verified purchase policy:

1. **Rating Validation:** Ensures rating is between 1 and 5
2. **Product Reviews:** Requires customer to have received (status='Delivered') the product
3. **Shop Reviews:** Requires customer to have completed purchase from the shop
4. **Fraud Prevention:** Prevents fake reviews from non-customers

Example Scenario:

```

1  -- Scenario 1: Valid product review (customer received product)
2  -- Customer 1 has Order 1 (status='Delivered') containing Product 1
3  INSERT INTO Review (customer_id, target_type, target_id, rating, comment)
4  VALUES (1, 'Product', 1, 5, 'Great product!');
5  -- Success
6
7  -- Scenario 2: Invalid review (customer never purchased product)
8  INSERT INTO Review (customer_id, target_type, target_id, rating, comment)
9  VALUES (2, 'Product', 1, 5, 'Fake review');
10 -- Error: 'Can only review products from delivered orders'
11
12 -- Scenario 3: Invalid rating
13 INSERT INTO Review (customer_id, target_type, target_id, rating, comment)
14 VALUES (1, 'Product', 1, 10, 'Rating too high');
15 -- Error: 'Rating must be between 1 and 5'
16
17 -- Scenario 4: Valid shop review
18 INSERT INTO Review (customer_id, target_type, target_id, rating, comment)
19 VALUES (1, 'Shop', 4, 4, 'Good service');
20 -- Success (Customer 1 bought from Shop 4)

```

10.2.2 Trigger: Review After Insert (Rating Update)

Purpose: Automatically updates product or shop rating after a new review is submitted.

Listing 49: Trigger: trg_review_after_insert

```

1  CREATE TRIGGER trg_review_after_insert
2  AFTER INSERT ON Review
3  FOR EACH ROW
4  BEGIN
5      DECLARE v_avg_rating DECIMAL(3, 2);
6
7      IF NEW.target_type = 'Shop' THEN

```

```

8      -- Update shop rating
9      SELECT AVG(rating) INTO v_avg_rating
10     FROM Review
11     WHERE target_type = 'Shop' AND target_id = NEW.target_id;
12
13     UPDATE Shop
14     SET rating = COALESCE(v_avg_rating, 0)
15     WHERE shop_id = NEW.target_id;
16
17     ELSEIF NEW.target_type = 'Product' THEN
18         -- Update product rating
19         SELECT AVG(rating) INTO v_avg_rating
20        FROM Review
21        WHERE target_type = 'Product' AND target_id = NEW.target_id;
22
23        UPDATE Product
24        SET rating = COALESCE(v_avg_rating, 0)
25        WHERE product_id = NEW.target_id;
26    END IF;
27 END$$

```

Trigger Type: AFTER INSERT

Related Tables: REVIEW, SHOP, PRODUCT

Business Logic: Maintains denormalized rating data for performance:

1. **Average Calculation:** Computes AVG(rating) from all reviews
2. **Shop Rating Update:** Updates Shop.rating for shop reviews
3. **Product Rating Update:** Updates Product.rating for product reviews
4. **Real-time Sync:** Rating immediately reflects new review

Example Scenario:

```

1  -- Before: Product 1 has 2 reviews (5 stars, 4 stars)
2  -- Product.rating = 4.50
3
4  -- Customer adds 3-star review
5  INSERT INTO Review (customer_id, target_type, target_id, rating, comment)
6  VALUES (3, 'Product', 1, 3, 'Average product');
7
8  -- After trigger execution:
9  -- Product 1 now has 3 reviews: (5 + 4 + 3) / 3 = 4.00
10 -- Product.rating = 4.00
11
12 SELECT rating FROM Product WHERE product_id = 1;

```

13 -- Result: 4.00

10.3 Inventory Management Triggers

10.3.1 Trigger: Product Item Before Update

Purpose: Validates business constraints before updating product item (prevents negative stock/price).

Listing 50: Trigger: trg_product_item_before_update

```
1 CREATE TRIGGER trg_product_item_before_update
2 BEFORE UPDATE ON ProductItem
3 FOR EACH ROW
4 BEGIN
5     -- Prevent negative stock
6     IF NEW.stock < 0 THEN
7         SIGNAL SQLSTATE '45000'
8         SET MESSAGE_TEXT = 'Product item stock cannot be negative';
9     END IF;
10
11     -- Prevent negative price
12     IF NEW.price < 0 THEN
13         SIGNAL SQLSTATE '45000'
14         SET MESSAGE_TEXT = 'Product item price cannot be negative';
15     END IF;
16 END$$
```

Trigger Type: BEFORE UPDATE

Related Tables: PRODUCTITEM

Business Logic: Enforces CHECK constraints at trigger level:

1. **Stock Constraint:** Prevents stock from becoming negative
2. **Price Constraint:** Prevents price from becoming negative
3. **Data Integrity:** Ensures business rules are enforced at database level

Example Scenario:

```
1 -- Scenario 1: Valid stock update
2 UPDATE ProductItem SET stock = 50 WHERE item_id = 1;
3 -- Success
4
5 -- Scenario 2: Attempt negative stock
6 UPDATE ProductItem SET stock = -10 WHERE item_id = 1;
```

```

7  -- Error: 'Product item stock cannot be negative'
8
9  -- Scenario 3: Attempt negative price
10 UPDATE ProductItem SET price = -1000 WHERE item_id = 1;
11 -- Error: 'Product item price cannot be negative'

```

10.4 Account Management Triggers

10.4.1 Trigger: Account After Insert (Subclass Creation)

Purpose: Automatically creates corresponding Customer/Shop/Admin record and related entities (Cart for Customer) when Account is inserted.

Listing 51: Trigger: trg_account_after_insert

```

1  CREATE TRIGGER trg_account_after_insert
2  AFTER INSERT ON Account
3  FOR EACH ROW
4  BEGIN
5      -- If customer, create customer record and cart
6      IF NEW.role = 'Customer' THEN
7          INSERT INTO Customer (customer_id, address, add_phone,
8                               total_spent, total_order)
9          VALUES (NEW.account_id, NULL, NULL, 0, 0);
10
11         INSERT INTO Cart (customer_id)
12         VALUES (NEW.account_id);
13
14         -- If shop, create shop record
15         ELSEIF NEW.role = 'Shop' THEN
16             INSERT INTO Shop (shop_id, shop_name, shop_phone,
17                              address_shop, rating)
18             VALUES (NEW.account_id, CONCAT('Shop ', NEW.account_id),
19                     NULL, NULL, 0);
20
21         -- If admin, create admin record
22         ELSEIF NEW.role = 'Admin' THEN
23             INSERT INTO Admin (admin_id, role, note)
24             VALUES (NEW.account_id, 'Support', NULL);
25     END IF;
26 END$$

```

Trigger Type: AFTER INSERT

Related Tables: ACCOUNT, CUSTOMER, SHOP, ADMIN, CART

Business Logic: Implements inheritance pattern (Account superclass):

1. **Customer Creation:** Creates Customer record + Cart for role='Customer'
2. **Shop Creation:** Creates Shop record with default name for role='Shop'
3. **Admin Creation:** Creates Admin record for role='Admin'
4. **Referential Integrity:** Ensures 1:1 relationship between Account and subclass

Example Scenario:

```

1  -- Create customer account
2  INSERT INTO Account (email, password, full_name, phone, role)
3  VALUES ('customer@email.com', 'hash123', 'John Doe',
4          '0123456789', 'Customer');
5  -- account_id = 10
6
7  -- After trigger execution:
8  -- Customer table: customer_id=10, total_spent=0, total_order=0
9  -- Cart table: cart_id=10, customer_id=10
10
11 -- Create shop account
12 INSERT INTO Account (email, password, full_name, phone, role)
13 VALUES ('shop@email.com', 'hash456', 'Shop Owner',
14         '0987654321', 'Shop');
15 -- account_id = 11
16
17 -- After trigger execution:
18 -- Shop table: shop_id=11, shop_name='Shop 11', rating=0

```

10.4.2 Trigger: Customer Before Insert (Code Generation)

Purpose: Auto-generates customer_code with CUST prefix (CUST00001, CUST00002, ...) based on customer_id.

Listing 52: Trigger: trg_customer_before_insert

```

1  CREATE TRIGGER trg_customer_before_insert
2  BEFORE INSERT ON Customer
3  FOR EACH ROW
4  BEGIN
5      -- Generate customer_code from customer_id
6      SET NEW.customer_code = CONCAT('CUST', LPAD(NEW.customer_id, 5, '0'));
7  END$$

```

Trigger Type: BEFORE INSERT

Related Tables: CUSTOMER

Business Logic: Implements auto-generated business key:

1. **Code Format:** CUSTxxxxx where xxxxx is zero-padded customer_id
2. **LPAD Function:** Pads ID with leading zeros to 5 digits
3. **Consistency:** Ensures all customer codes follow same format

Example Scenario:

```
1  -- Insert customer with ID 1
2  INSERT INTO Customer (customer_id, address, total_spent, total_order)
3  VALUES (1, '123 Main St', 0, 0);
4
5  -- After trigger: customer_code = 'CUST00001'
6
7  -- Insert customer with ID 25
8  INSERT INTO Customer (customer_id, address, total_spent, total_order)
9  VALUES (25, '456 Elm St', 0, 0);
10
11 -- After trigger: customer_code = 'CUST00025'
12
13 -- Insert customer with ID 1000
14 INSERT INTO Customer (customer_id, address, total_spent, total_order)
15 VALUES (1000, '789 Oak St', 0, 0);
16
17 -- After trigger: customer_code = 'CUST01000'
```

10.4.3 Trigger: Shop Before Insert (Code Generation)

Purpose: Auto-generates shop_code with SHOP prefix (SHOP00001, SHOP00002, ...) based on shop_id.

Listing 53: Trigger: trg_shop_before_insert

```
1  CREATE TRIGGER trg_shop_before_insert
2  BEFORE INSERT ON Shop
3  FOR EACH ROW
4  BEGIN
5      -- Generate shop_code from shop_id
6      SET NEW.shop_code = CONCAT('SHOP', LPAD(NEW.shop_id, 5, '0'));
7  END$$
```

Trigger Type: BEFORE INSERT

Related Tables: SHOP

Business Logic: Same pattern as customer_code generation:

1. **Code Format:** SHOPxxxxx where xxxxx is zero-padded shop_id
2. **LPAD Function:** Pads ID with leading zeros to 5 digits
3. **Business Key:** Provides user-friendly identifier for shops

Example Scenario:

```
1  -- Insert shop with ID 4
2  INSERT INTO Shop (shop_id, shop_name, rating)
3  VALUES (4, 'Tech Store', 0);
4
5  -- After trigger: shop_code = 'SHOP00004'
6
7  -- Insert shop with ID 100
8  INSERT INTO Shop (shop_id, shop_name, rating)
9  VALUES (100, 'Fashion Hub', 0);
10
11 -- After trigger: shop_code = 'SHOP00100'
12
13 SELECT shop_id, shop_code, shop_name FROM Shop;
14 -- Results:
15 -- 4 | SHOP00004 | Tech Store
16 -- 100 | SHOP00100 | Fashion Hub
```

11 Application Demo and System Workflow

This section demonstrates the complete e-commerce system architecture and how the application works from database to user interface.

11.1 System Architecture Overview

11.1.1 Three-Tier Architecture Design

1. Data Layer (MySQL Database): The database layer handles all data storage and business logic execution:

- **14 Tables:** Account (superclass), Customer, Shop, Admin (inheritance), Product, ProductItem, Category (recursive hierarchy), Order, OrderItem, Cart, CartItem, Voucher, Shipping, Review
- **6 Functions:** Calculate totals (order total, customer spent, shop revenue), Calculate ratings (product rating, shop rating), Cursor demonstration (order total items)
- **13 Procedures:**

- Category hierarchy navigation (get hierarchy, get all children, get all parents) - using Recursive CTE
- Order management (create order from cart, apply voucher) - complex transactions
- Shop analytics (revenue report with top products) - aggregations and reporting
- Product CRUD (add, update, delete product and product items) - data manipulation
- Product search (dynamic filtering, sorting, pagination) - advanced queries
- **9 Triggers:** Auto-generate customer/shop codes, manage inventory stock, update rating statistics, create cart automatically, enforce business rules

2. Application Layer (Node.js + Express): The backend provides RESTful API endpoints organized by business modules:

- **MVC Architecture:** Controllers handle requests, call database procedures/functions, return JSON responses
- **11 Controller Modules:**
 - `auth.controller.js`: User registration, login, JWT token management
 - `product.controller.js`: Product search, CRUD operations, variant management
 - `cart.controller.js`: Add/remove items, update quantities, view cart
 - `order.controller.js`: Create orders, view history, update status, cancellation
 - `customer.controller.js`: Profile management, order history, statistics
 - `shop.controller.js`: Shop information, revenue reports, analytics
 - `review.controller.js`: Submit reviews, view ratings, calculate averages
 - `category.controller.js`: Category tree, parent-child relationships
 - `voucher.controller.js`: Validate codes, apply discounts
 - `shipping.controller.js`: Shipping methods, fee calculation
 - `admin.controller.js`: System management, user administration
- **Middleware:** JWT authentication, role-based authorization, error handling, request validation
- **Routes:** RESTful endpoints following naming conventions (GET `/api/products`, POST `/api/orders`, etc.)

3. Presentation Layer (React + Redux): The frontend provides interactive user interfaces for different user roles:

- **Page Structure:**
 - **Public Pages (5):** HomePage, SignInPage, SignUpPage, ProductListPage, ProductDetailPage

- **Customer Pages** (6): CartPage, CheckoutPage, OrderConfirmationPage, OrderDetailPage, ProfilePage, ReviewPage
- **Shop Owner Pages** (4): ShopDashboard, ShopProducts, ShopOrders, ShopAnalytics
- **Admin Pages** (3): AdminDashboard, UserManagement, SystemReports
- **Components:** Reusable UI components (Button, Modal, Card, Input, Dropdown, Spinner, Toast notifications)
- **State Management:** Redux stores for auth, cart, products, orders - synchronized across all pages
- **Routing:** React Router with protected routes based on user roles

11.2 Main Features Demonstration

11.2.1 Feature 1: Product Search and Discovery

User Experience:

- Customer visits homepage, browses category menu (Electronics > Phones)
- Enters search term "iPhone" in search bar
- Applies filters: Price range (0-2,000,000 VND), Rating (4+ stars)
- Sorts results by rating (descending)
- Views paginated results showing 20 products per page

Behind the Scenes:

- Frontend sends GET request: `/api/products?search=iPhone&category=3&minPrice=0&maxPrice=`
- Backend calls: `CALL sp_get_product_list(...)`
- Database executes dynamic SQL with JOINS (Product, ProductItem, Shop, Review)
- Calculates: MIN/MAX price, total stock, variant count, average rating
- Returns JSON with products array and pagination metadata
- Frontend renders product grid with images, prices, ratings, stock status

11.2.2 Feature 2: Shopping Cart Management

User Experience:

- Customer selects product variant (iPhone 15, Red, 128GB)
- Clicks "Add to Cart" button

- Toast notification confirms: "Product added to cart!"
- Cart icon updates showing item count
- Navigates to CartPage to review items
- Adjusts quantities using +/- buttons
- Removes unwanted items
- Sees real-time subtotal calculation

Behind the Scenes:

- Frontend: POST `/api/cart/add` with `{item_id, quantity}`
- Backend: INSERT into CartItem or UPDATE quantity if exists
- Database: Retrieves cart_id from Cart table (one cart per customer)
- Redux store updates cart state globally
- All components showing cart info re-render automatically

11.2.3 Feature 3: Order Creation with Voucher**User Experience:**

- Customer proceeds to checkout
- Selects shipping method (Standard Shipping - 5,000 VND)
- Enters voucher code "DISCOUNT10" and clicks "Apply"
- System validates voucher: "10% discount applied! Save 220,000 VND"
- Confirms shipping address and payment method (Banking)
- Reviews final total: Subtotal (2,200,000) + Shipping (5,000) - Discount (220,000) = 1,985,000 VND
- Clicks "Place Order"
- Redirected to order confirmation page with order ID

Behind the Scenes:

- Voucher validation: CALL `sp_apply_voucher(code, amount, @discount, @valid, @message)`
- Checks: voucher exists, is active, not expired, usage limit not reached, order meets minimum value
- Order creation: CALL `sp_create_order_from_cart(...)`

- Database transaction executes:
 1. Validates customer, shipping method, cart has items
 2. Calculates subtotal from CartItems
 3. Applies voucher discount, adds shipping fee
 4. Inserts Order with status 'Processing'
 5. Moves CartItems to OrderItems
 6. Reduces ProductItem stock (inventory management)
 7. Clears cart
 8. Updates Customer statistics (total_order++, total_spent+=amount)
- Frontend receives order_id, updates Redux, shows confirmation

11.2.4 Feature 4: Product Review and Rating

User Experience:

- Customer receives order with status "Delivered"
- Navigates to OrderDetailPage, sees "Leave Review" button
- Opens ReviewPage for Product 1 (iPhone 15)
- Selects 5-star rating
- Writes comment: "Excellent product! Fast delivery, highly recommend!"
- Clicks "Submit Review"
- Success message: "Review submitted! Product rating updated to 4.67 stars"

Behind the Scenes:

- Frontend: POST /api/reviews/add with {target_id, target_type, rating, comment}
- Backend: INSERT INTO Review
- **Trigger fires automatically:** trg_review_after_insert
- Trigger calls function: fn_calculate_product_rating(product_id)
- Function calculates: **AVG(rating)** from all product reviews
- Example: Previous reviews (5, 4), New review (5) $\rightarrow$ Average = $(5+4+5)/3 = 4.67$
- Trigger updates: UPDATE Product SET rating = 4.67
- All users immediately see updated rating on product pages

11.2.5 Feature 5: Shop Revenue Analytics

User Experience:

- Shop owner logs in, navigates to ShopAnalytics page
- Selects date range: January 1, 2025 - December 31, 2025
- Clicks "Generate Report"
- Views dashboard showing:
 - Total revenue: 1,980,000 VND
 - Total orders: 1
 - Total units sold: 2
 - Average order value: 1,980,000 VND
 - Top 10 selling products with breakdown by variant
- Exports report to PDF for accounting

Behind the Scenes:

- Frontend: GET /api/shops/4/revenue?from=2025-01-01&to=2025-12-31
- Backend: CALL sp_get_shop_revenue_report(shop_id, from_date, to_date)
- Procedure returns two result sets:
 1. Shop summary with aggregated metrics
 2. Top 10 products ordered by revenue (DESC)
- Frontend renders charts (revenue trend graph, product pie chart)

12 References and Video Demonstration

12.1 GitHub Repository

The complete project source code is available on GitHub: [Link Github](#): [Link Github](#)

12.2 Video Demonstration

A comprehensive video demonstration of the application is available. The video covers: Video demo: [HERE](#)

13 Conclusion

This report has presented a complete full-stack implementation of the Shoppii e-commerce platform, integrating database design, backend API, and frontend interface.

13.1 Summary

The project demonstrates:

- **Database (MySQL):** 13 tables, 6 functions, 13 procedures, 9 triggers with advanced SQL techniques (Recursive CTE, transactions, automatic triggers)
- **Backend (Node.js/Express):** 11 controllers with 50+ RESTful endpoints, JWT authentication, role-based access control
- **Frontend (React/Vite):** 19+ pages, Redux state management, real-time updates, responsive UI components
- **Integration:** Seamless workflow from user interface through API to database procedures with automatic trigger coordination

13.2 Key Features

- Multi-role platform: Customers, Shop Owners, Admins
- Product search with dynamic filtering and sorting
- Shopping cart and checkout with voucher support
- Order processing with automatic stock management
- Review system with automatic rating calculation
- Shop analytics and revenue reporting
- Transaction safety with ACID compliance